



第一章操作系统概述

1. OS的概念：系统软件、资源管理者、使用接口
2. OS的发展历史：从无到有、从弱到强
3. OS的类型：批处理、分时、实时、NOS、DOS、嵌入式
4. OS开发的目标和原则
5. OS的结构：宏内核、微内核、虚拟机...
6. 几个典型的操作系统：UNIX、Windows、Linux、macOS/iOS、Android、中标麒麟、7个公司8个人
7. openEuler操作系统、HarmonyOS
8. 重要的概念：核心态、用户态、并行、并发、多道程序设计、宏内核、微内核、虚拟机



第二章

用户接口与作业管理





内容目录



2.1 程序的启动和结束

2.2 作业（JOB）的基本概念

2.3 作业的建立

2.4 用户接口

2.5 系统调用

2.6 Linux系统调用

本章目标 (Objectives) :

1. 了解程序的几种启动方式，能够利用相应的方式启动程序；
2. 了解作业的基本概念
3. 了解作业输入的几种方式
4. 理解Spooling系统的工作原理
5. 理解系统调用的工作原理，能够利用系统调用完成用户的需求
6. 能够正确区分API和系统调用
7. 了解Linux的系统调用的实现



2.1 程序的启动和结束



2.1.1 程序的启动

● 程序开始执行时必须满足

- ① 程序已装入内存，对吗？
- ② 程序计数器PC（或者是IP）中已置入该程序在内存的入口地址，即CS:IP指向了该程序在内存的入口地址

- 有问题吗？
- 为什么是这样？
依据是什么？

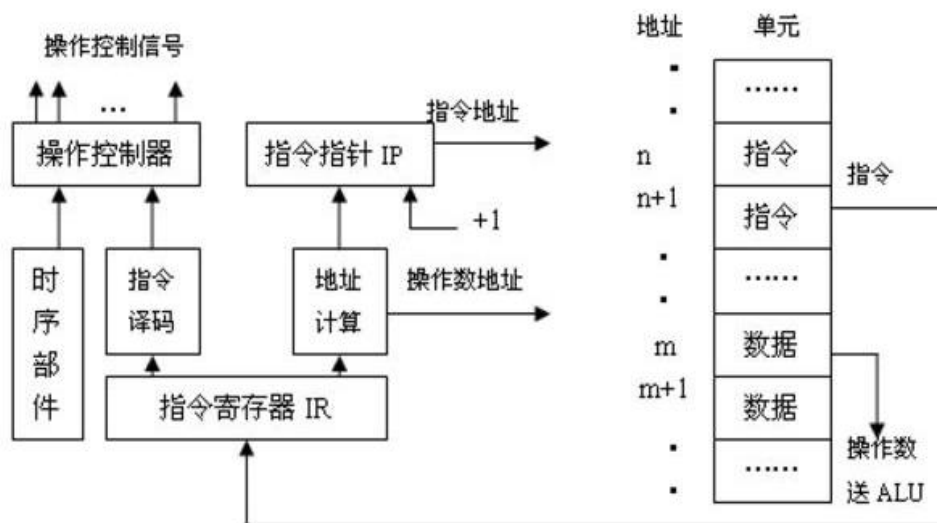


图 1-8 控制器工作原理图



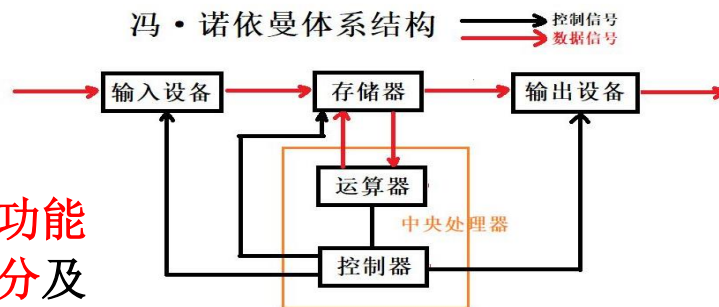
2.1 程序的启动和结束



2.1.1 程序的启动

计算机体系结构

- 计算机体系结构是指根据**属性和功能不同**而划分的计算机理论**组成部分**及计算机基本**工作原理、理论**的总称。其中计算机**理论组成部分**并不单与**某一个实际硬件相挂钩**，如存储部分就包括寄存器、内存、硬盘等。



<https://blog.csdn.net/waver>



冯·诺依曼体系工作原理(CPU工作原理)

冯·诺依曼体系结构

- 冯·诺伊曼是美籍匈牙利数学家，**数字计算机之父**。
- 冯·诺伊曼于1946年提出**存储程序原理**：程序本身当作数据来对待，程序和它要处理的数据用同样的方式储存，并确定了存储程序计算机的五大组成部分和基本工作方法。
- 冯·诺依曼理论**要点是：计算机的数制采用**二进制**；计算机应该**按照程序顺序执行**。人们把冯·诺伊曼的这个理论称为**冯·诺伊曼体系结构**。
- 从**ENIAC**到当前**最先进的计算机**都采用的是冯·诺伊曼体系结构。



2.1 程序的启动和结束



2.1.1 程序的启动

● 问题：程序有几种启动(或运行)方式？

- 命令方式（GUI的点击）、批处理方式、EXEC方式、自启动方式

● 第一种方式：命令方式

- 命令提示符下输入程序名和参数，回车

命令提示符：c>, \$, %

- 命令解释程序

command.com （MS-DOS的）

SHELL （如UNIX或Linux的Bash, Csh, Ksh...）

Windows：窗口菜单显示和鼠标操作



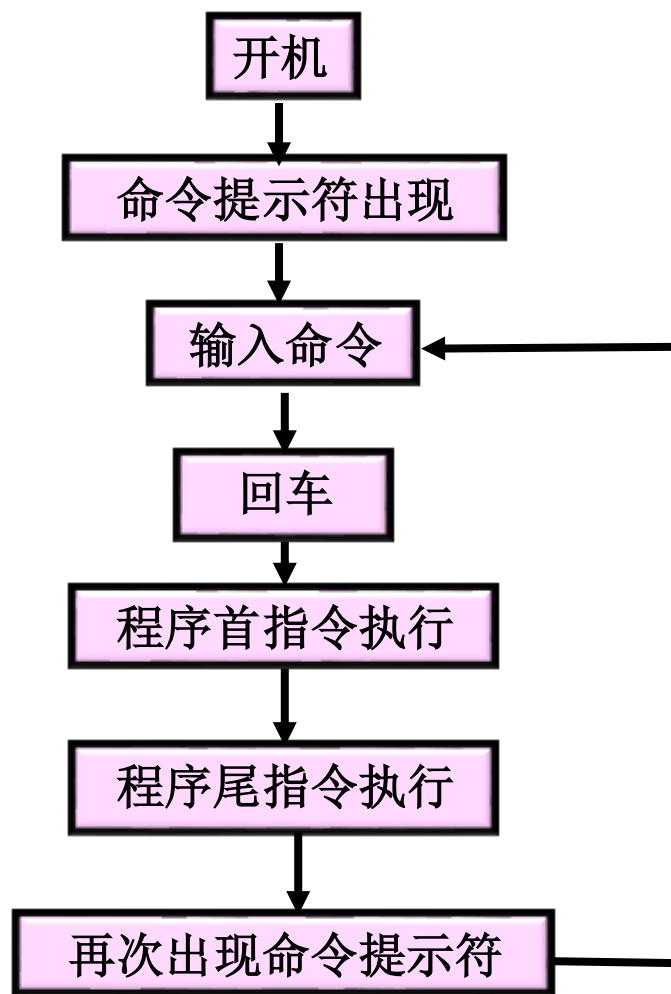
2.1 程序的启动和结束



2.1.1 程序的启动

● 第一种方式：命令方式

```
命令提示符
2019/12/07 17:09 809,472
Bubbles.scr
2021/01/10 08:49 67,072
BWContextHandler.dll
2021/12/20 12:23 90,624
ByteCodeGenerator.exe
103 个文件 35,067,9
90 字节
3 个目录 182,871,912,
448 可用字节
c:\Windows\System32>cala
'cala' 不是内部或外部命令，也不是可
运行的程序
或批处理文件。
c:\Windows\System32>calc
c:\Windows\System32>
```





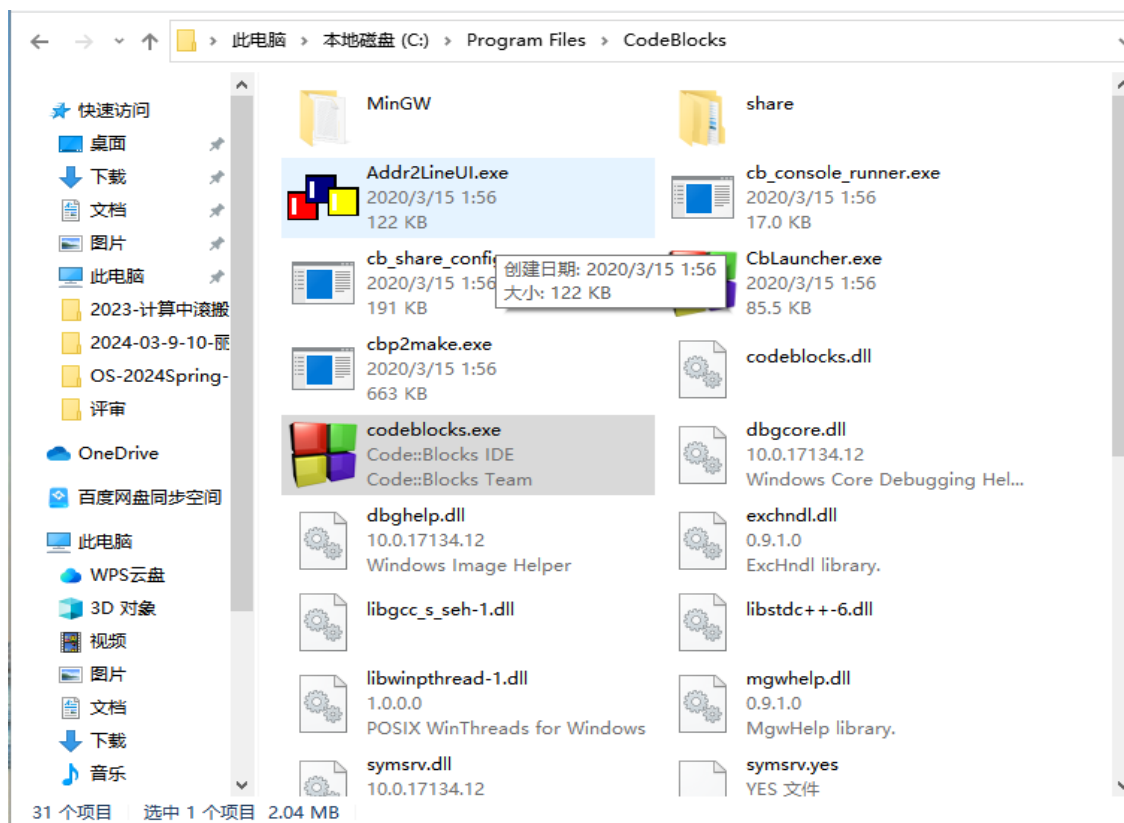
2.1 程序的启动和结束



2.1.1 程序的启动

(1) 命令方式

● 双击图标方式运行命令或应用程序





2.1 程序的启动和结束



2.1.1 程序的启动

(2) 批处理方式

- 把一些需要连续执行多条命令才能完成的任务，放在一个批处理文件或者脚本文件中
- 例：Windows环境下，开机后启动QQ、Gtalk、旺旺、浏览器
- 建立批处理文件：Gtalk+QQ+WangWang.bat

```
@echo off
echo Starting QQ...
start "" D:\Program Files\QQ\qq.exe
echo Starting WangWang...
start "" D:\Program Files\WangWang\WangWang.exe
echo Starting Google Talk...
start "" D:\Program Files\Gtalk\googletalk.exe
```



2.1 程序的启动和结束



windows操作系统下的msdtcvttr.bat

```
0 10 20 30 40 50 60 70 80 90 100 110 120 130
1  @echo off
2
3  rem ***** start of 'main'
4
5  set DEBUG=0
6  if "%DEBUG%"=="1" (set TRACE=echo) else (set TRACE=rem)
7
8  rem Note that right now there is a bug in tracerpt.exe cause of which you might want to use tracefmt.exe instead.
9  rem set the value 1 if you want to use tracefmt.exe instead
10 set USE_TRACE_FMT=1
11 %TRACE% The value of variable USE_TRACE_FMT is %USE_TRACE_FMT%
12
13 rem define variables for each of the switches we support
14 set SWHELP=h
15 set SWMODE=mode
16 set SWTRACELOG=tracelog
17 set SWMOF=mof
18 set SWOUTPUT=o
19 set VALID=0
20
21 rem define variables for parameters to be passed to tracerpt
22 set TRACEDIR=%WINDIR%\system32\msdtc\trace
23 set TRACEFILE1=%TRACEDIR%\dtctrace.log
24 set TRACEFILE2=%TRACEDIR%\tracetx.log
25 set MOFFILE=%TRACEDIR%\msdtctr.mof
26 set ERRORFILE=%TRACEDIR%\errortrace.txt
27 set OUTPUTFILE=%TRACEDIR%\trace
28
29 rem Parse command line and setup variables
30 set CMDLINE=%*
31 %TRACE% About to call PARSECMDLINE with the argument %CMDLINE%
32 call :PARSECMDLINE 0
33
```



2.1 程序的启动和结束



2.1.1 程序的启动

(3) EXEC方式

- ✓ 在一个程序中运行另一个程序
- ✓ 如：MS-DOS或者Linux下的EXEC调用。例：在应用程序执行中先清屏，然后在输出结果，怎么办？
- ✓ 在Linux/Unix下，创建进程使用fork/exec方式
- ✓ 高级语言中提供的exec族函数：

/*启动某路径下的可执行程序，并可以向该可执行程序传递参数*/

(1)int execl (**const char *path**,**const char *arg0**,char *arg1,...,char *argn,NULL)

(2)int execl(**const char *path**,char *arg0,char *arg1,...,char *argn,NULL,**char *const envp[]**) /*启动某路径下的可执行程序，并将新的**环境变量**值赋给该可执行程序.*/

(3)int execlp(**const char *filename**,char *arg0,char *arg1,...,NULL)

(4)int **execv** (**const char *path**,**char *const argv[]**)

(5)int **execve**(**const char *path**,**char *const argv[]**,**char *const envp[]**)

(6)int **execvp**(**const char *filename**,**char *const argv[]**)



2.1 程序的启动和结束



2.1.1 程序的启动

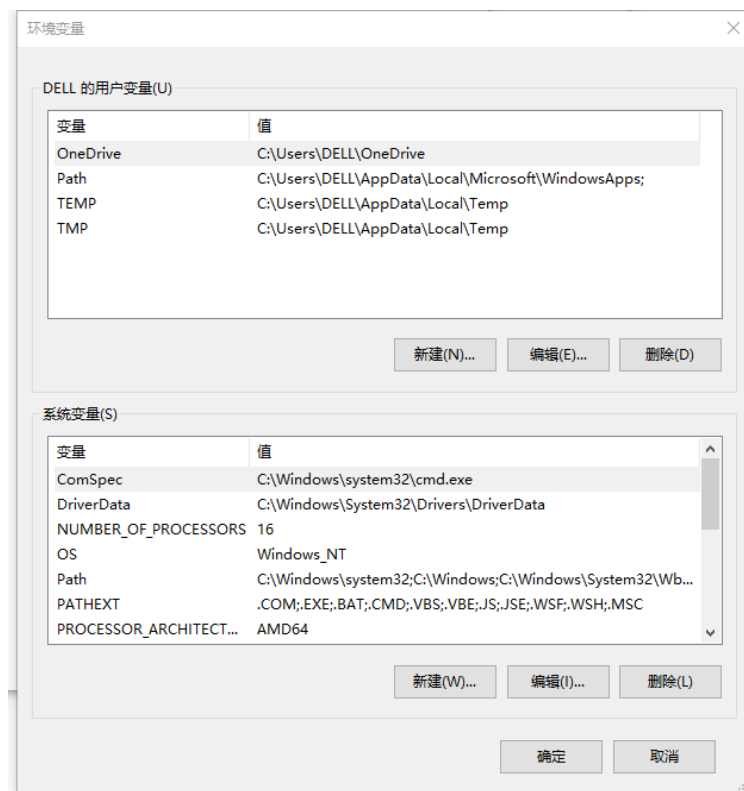
(3) EXEC方式

概念：环境变量(environment variables)

一般是指在OS中用来指定OS运行环境的一些参数，如**临时文件夹位置、系统文件夹位置**等。例如：

- **%OS%**：返回操作系统的名称。
- **%PATH%**：指定可执行文件的搜索路径。
- **%PATHEXT%**：返回操作系统认为可执行的文件扩展名的列表。

右键开始菜单图标-->系统-->高级系统设置





例：利用exec启动运行其它程序



/* 一个简单的利用exec调用操作系统本身命令，构造的shell */

```
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
char command[256];
void main() {
    int rtn; /*子进程的返回数值*/
    while(1) {
        /* 从终端读取要执行的命令 */
        printf( ">" );
        fgets( command, 256, stdin );
        command[strlen(command)-1] = 0;
```

```
        if ( fork() == 0 ) { /* 子进程执行此命令 */
            execlp( command, NULL );
            /* 如果exec函数返回，表明没有正常执行
            命令，打印错误信息*/
            perror( command );
            exit( errno );
        }
        else { /* 父进程， 等待子进程结束，并打印
        子进程的返回值 */
            wait ( &rtn ); //如果不调用该语句，是什
                           么样的表现？
            printf( " child process return %d\n", rtn );
        }
    } /*end while*/
}
```



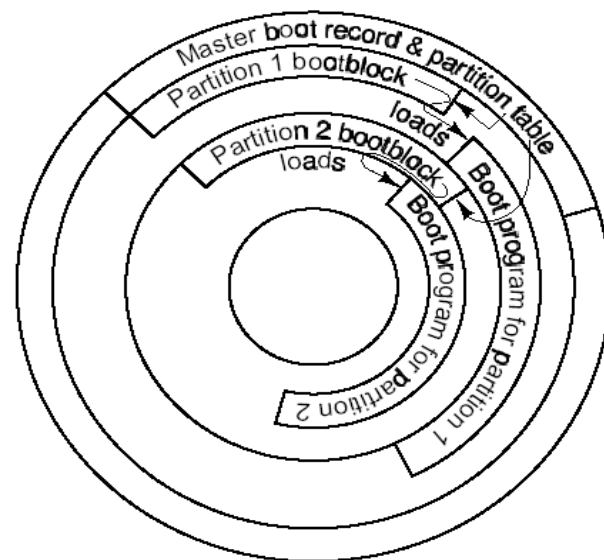
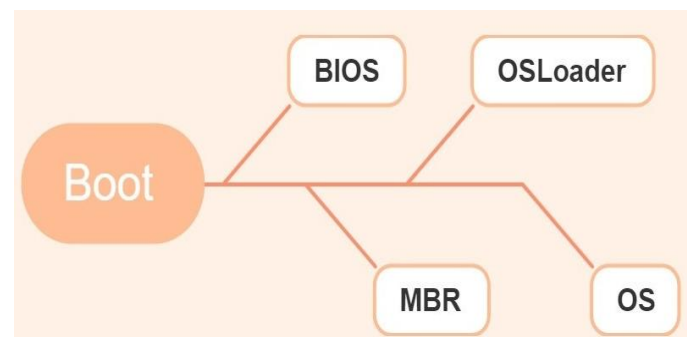
2.1 程序的启动和结束



2.1.1 程序的启动

(4) 自启程序

- 自己装入自己，并启动自己开始执行的程序
- 自启程序由两部分组成
 - ✓ 引导程序
 - ✓ 程序主体
- OS的引导过程(bootstrap)
 - ✓ 开机加电，跳到主板的固定位置的ROM **BIOS**中执行，把引导盘中的引导扇区装入内存，然后执行这段程序，再将OS程序和数据读进来，执行。



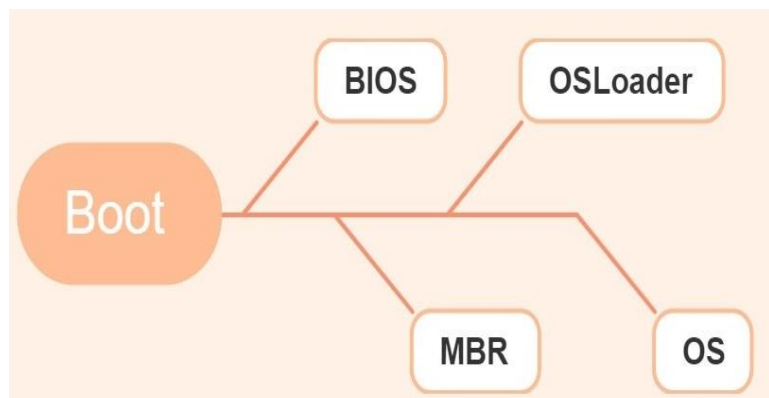


2.1 程序的启动和结束

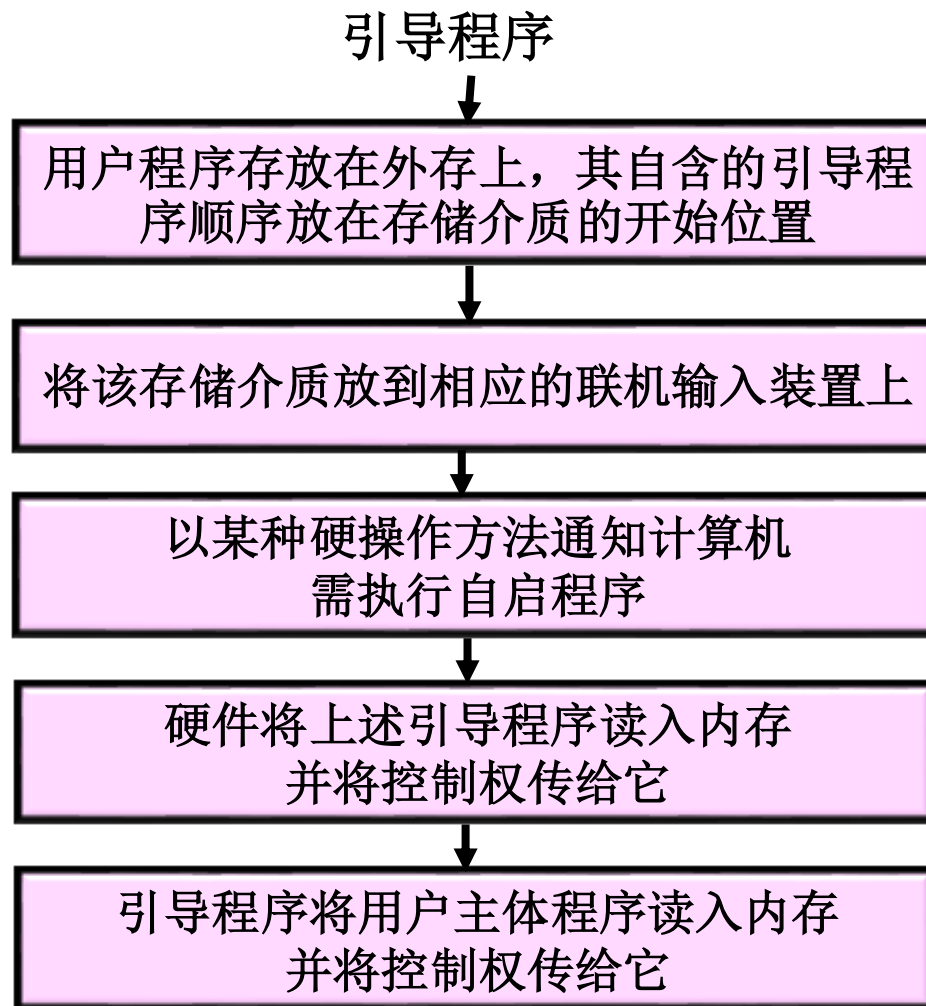


2.1.1 程序的启动 (4) 自启程序

计算机启动的几个阶段



源自: <https://zhuanlan.zhihu.com/p/370535266>





2.1.1 程序的启动

(4) 自启程序--OS引导过程定义

- OS的引导过程是指：计算机启动时，由系统**BIOS**以及**OS引导程序**将OS内核可执行代码**逐级**装入内存并开始执行，直到系统控制台显示“Login:”登录提示为止的系统引导阶段。
- 不同体系结构的机器，其引导过程存在差异，但原理相通。下面介绍OS在IBM PC及其兼容机上的启动过程。

- 如何实现的？

按下电源



BIOS自检

系统引导
lilo/grub

启动内核

初始化系统

```
CentOS Linux release 6.0 (Final)
Kernel 2.6.32-71.el6.i686 on an i686
maple login: _
```

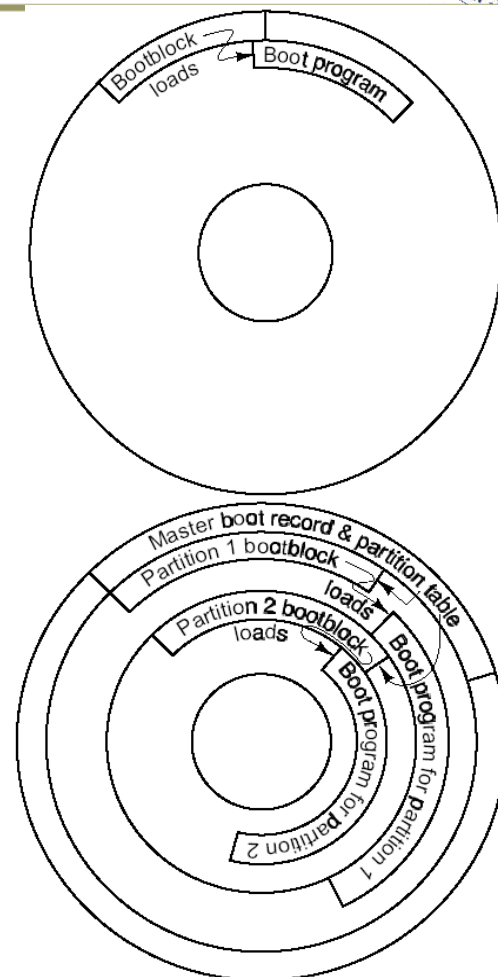



2.1 程序的启动和结束



I386芯片的开机启动到BIOS启动

- 系统上电或RESET时，主板送出低电平的“Power Good”信号，使CPU初始化；
- 之后，进入**实地址模式**。此时 CS:EIP=F000:FFF0H，DS=ES=SS=FS=GS=0000H，在实模式下，**CPU从CS:EIP处的指令执行**；
- BIOS的第一条指令是跳转指令。BIOS两个任务：
 - 一是用于**测试系统资源 (POST)**
 - 负责将**物理0面、0道、1扇区的内容 (Bootlorder)**装入内存中**0x7C00处**，然后转此处执行。



注：在IBMPC中，考虑向下兼容，BIOS服务程序存储在主板的ROM中，占1M内存的F0000H~FFFFFH；BIOS独立于OS，即不论运行DOS、Windows、Linux、Minix，在开机时都调用ROM BIOS中的服务来加载系统引导程序。



2.1 程序的启动和结束

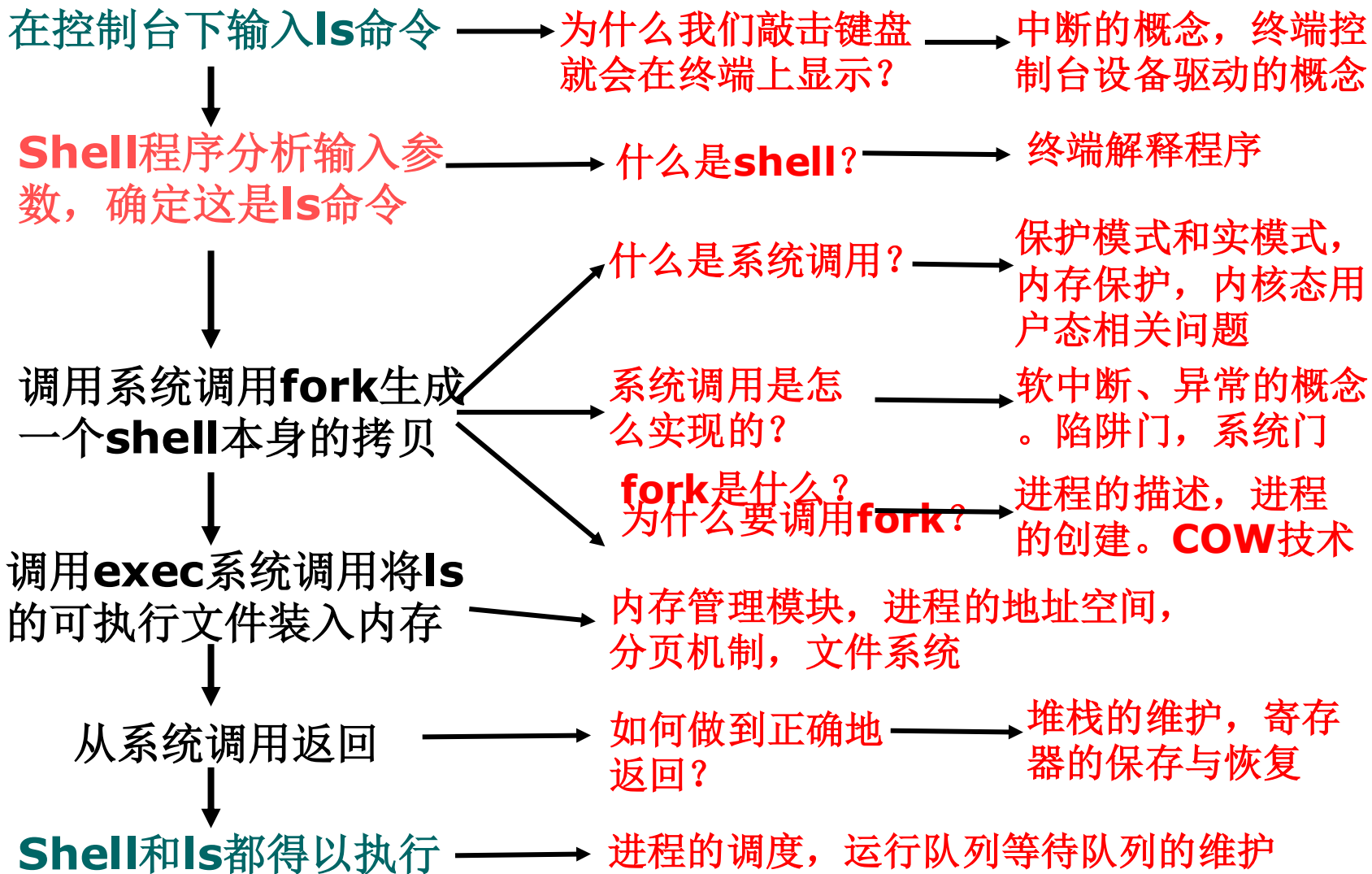


2.1.2 程序的结束

- **正常结束：** 程序按自身的逻辑有效地完成预定功能后结束
 - 高级语言主程序，例如C语言的main，执行的最后，会**隐形地**调用**exit()**函数做退出处理
 - **释放所用资源**（空间、设备），记录使用情况，记帐等
 - 给父程序发消息，并回送结果信息
- **异常结束：** 发生了某些错误而导致程序在没有完成预定功能时提前结束



一条命令的艰辛执行历程【自学】





2.1 程序的启动和结束



小结:

- 程序的启动方式:
 - ✓ 4种方式
 - ✓ 关注exec启动方式
- Q&A



```

graph LR
    A[编程输入] <--> B[调试]
    B <--> C[编译]
    C --> D[链接]
    D --> E[执行]
    E --> F[输出]
    C --> G(目标程序段)
    D --> H(可执行程序)
  
```

-
- ```

graph LR
 A[源程序] --> B[编译]
 B --> C[(目标程序)]
 C --> D[链接装配]
 E((子程序)) --> D
 F((库函数)) --> D
 D --> G[(执行程序)]
 G --> H[运行]
 I[输入数据] --> H
 H --> J[输出信息]
 H --> K[运行结果]
 J -.-> B
 J -.-> D

```

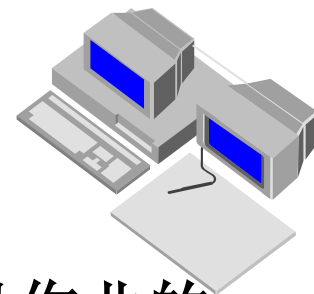


## 2.2 作业 (JOB) 的基本概念



### 2.2.1 作业 (从系统角度)

- 作业 = 程序 +  
数据 (作业体) +  
作业说明书 (作业控制语言) ;



- 实例:

- 云计算环境下, 计算任务的提交, 就是以作业的方式
- 在批处理系统中, 作业是 **占用内存** 的基本单位, 即以作业为单位将程序和数据调入内存。



## 2.2 作业 (JOB) 的基本概念



### 2.2.2 作业组织

● 作业=程序+数据+作业说明书

● 作业说明书

- 体现用户的控制意图, 应该包含什么信息?
- 包括作业基本情况、作业控制、作业资源要求的描述
  - ✓ 作业基本情况: 用户名、作业名、编程语言、最大处理时间等
  - ✓ 作业控制描述: 作业控制方式、作业步的操作顺序、作业执行出错处理
  - ✓ 作业资源要求描述: 处理时间、优先级、内存空间、外设类型和数量等
- 它由作业控制语言编写



## 2.2 作业 (JOB) 的基本概念



### 2.2.2 作业组织

#### ● 作业控制语言

- 用户用于描述批处理作业处理过程控制意图的一种特殊程序
- 书写作业说明书的语言称为**作业控制语言** (JCL)
- 例如：批处理文件或shell







## 2.3 作业的建立



- **作业的建立**：一个作业的全部程序和数据输入到外存，且在系统中建立相应的**作业控制块（Job Control Block, JCB）**；
- **作业建立分两个阶段**：**作业输入、JCB建立**；
- **作业的输入**：
  - 将作业的程序、数据和作业说明书从输入设备**输入到外存**，并形成有关初始信息；
  - 必须有**外部启动信号**通知系统调用相应的**输入管理程序**—决定了**作业的输入方式**。

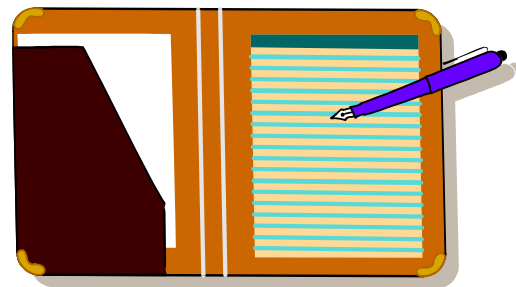


## 2.3 作业的建立



### 2.3.1 作业输入方式

- 联机输入方式
- 脱机输入方式
- 直接耦合方式
- Spooling方式
- 网络输入方式





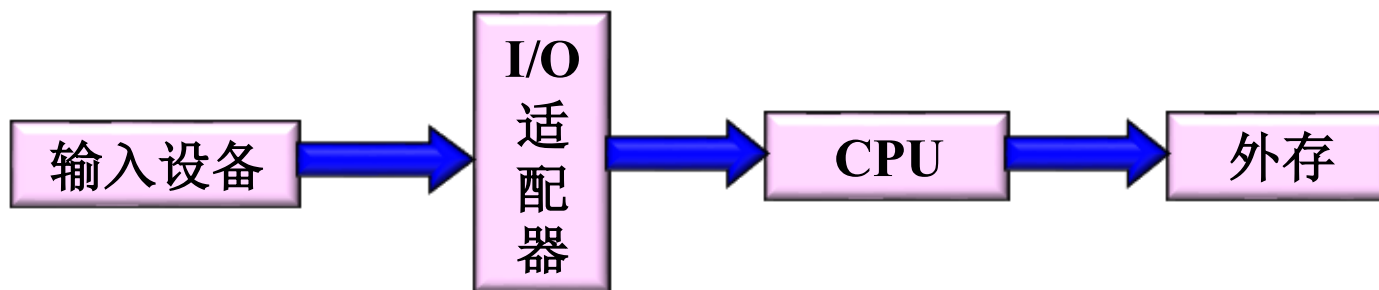
## 2.3 作业的建立



### 2.3.1 作业输入方式

#### (1) 联机输入方式

- 联机输入方式：外围设备和主机直接连接，又称预输入方式



- 单台设备和主机连接时，I/O设备与作业处理不能并行；降低了CPU效率；
- 多台外设同时联机输入——SPOOLING系统。



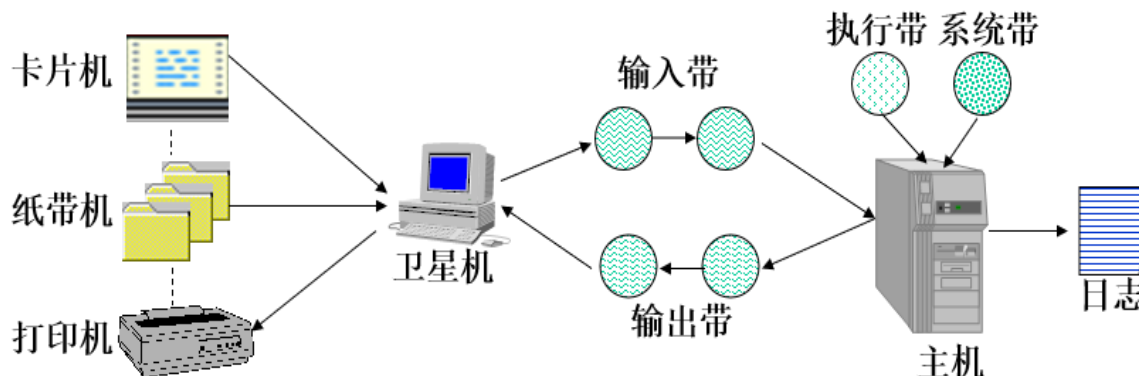
## 2.3 作业的建立



### 2.3.1 作业输入方式

#### (2) 脱机输入方式

- 脱机输入方式：利用低档PC机作为外围处理机进行输入处理；
- 在个人机上，用户通过联机方式将作业输入到**后援存储器**，然后将装有输入数据**的后援存储器拿到**主机的高速外设上与主机连接。



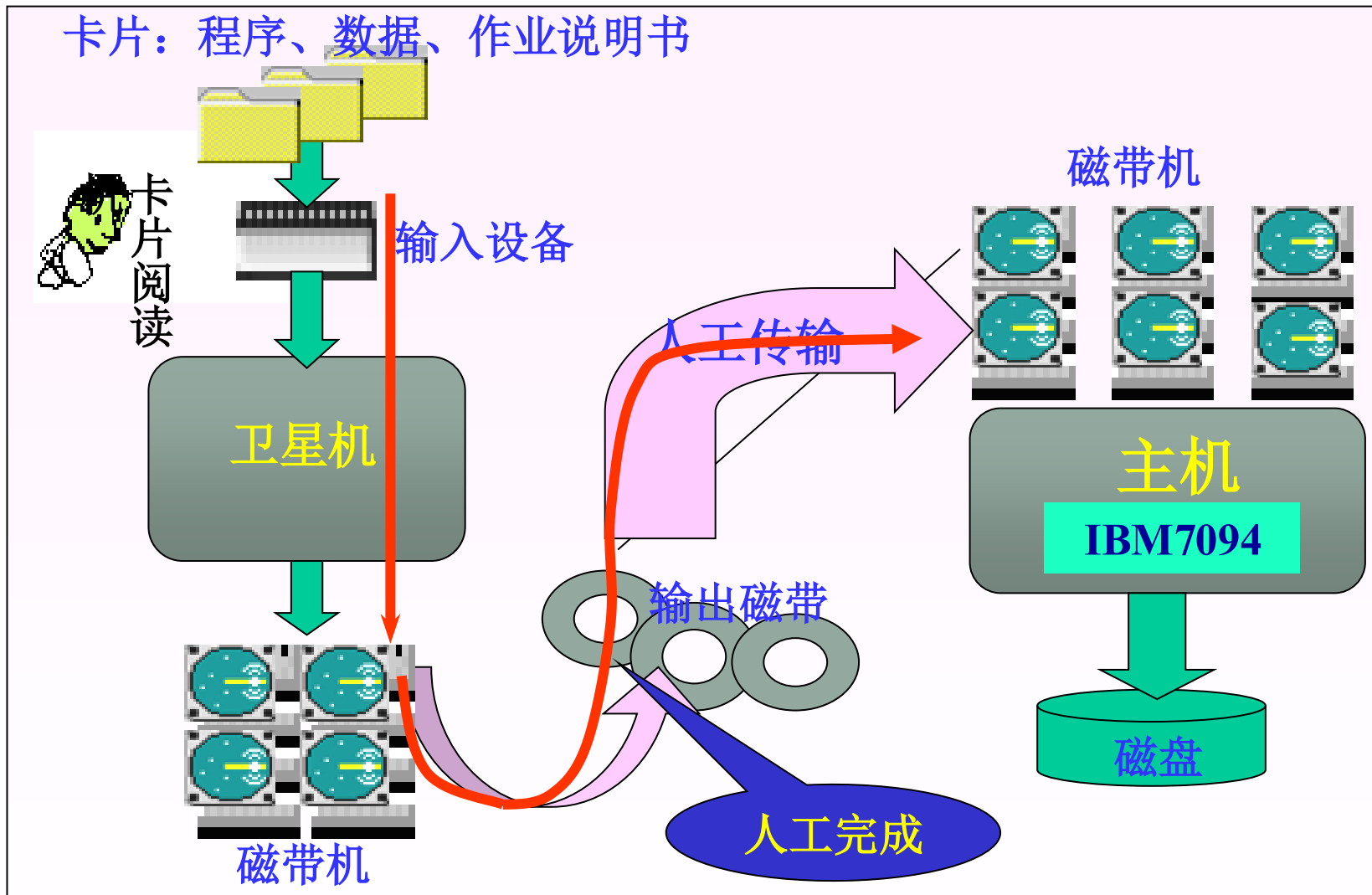
#### ● 特点:

- ✓ 解决了主机CPU的浪费，以牺牲个人机为代价；
- ✓ 灵活性差，需人工干预介质传送，不安全。



## 2.3 作业的建立

脱机输入方式实例





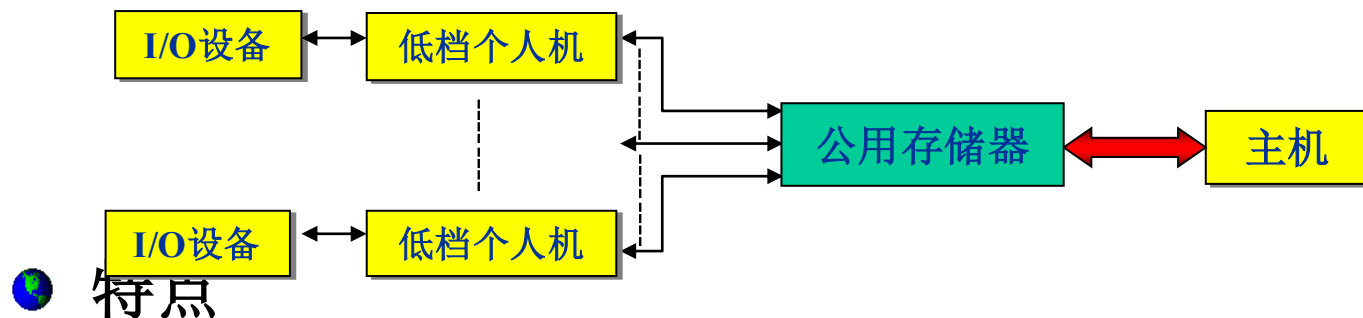
## 2.3 作业的建立



### 2.3.1 作业输入方式

#### (3) 直接耦合方式

- 直接耦合方式：将主机和外围低档机通过一个**公用大容量外存**直接耦合



#### ● 特点

- 保留了脱机方式快速的优点，克服了其人工干预的缺点；
- 需要大容量公用存储器和多台低档机，成本高。



## 2.3 作业的建立



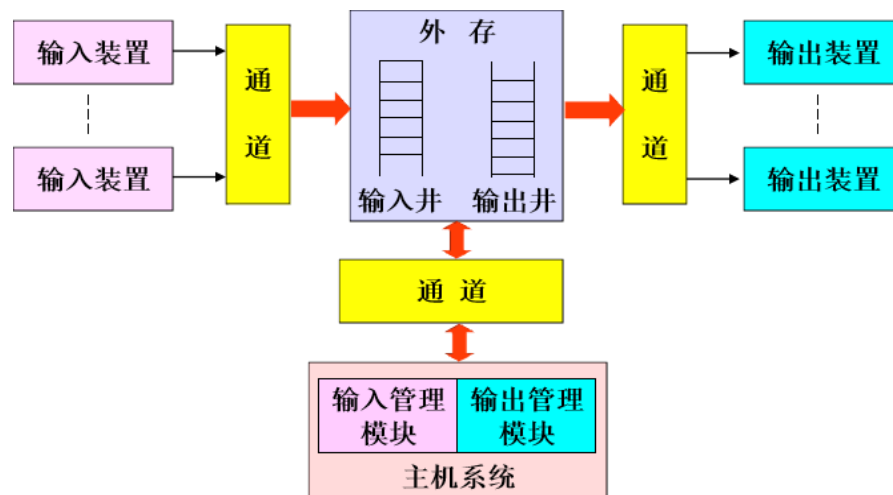
### 2.3.1 作业输入方式

#### (4) SPOOLING系统

(Simultaneously Peripheral Operation On Line-同时外围设备联机操作)

- SPOOLING系统：假脱机系统把作业处理的全过程划分为相对独立的三个部分--输入流、处理流和输出流
- spooling-in/ spooling-out进程：控制输入/输出，包括输入程序模块、输出程序模块、作业调度模块

**通道 (Channel)**：一个独立于CPU的专门I/O控制的处理机，控制设备与内存直接进行数据交换。它有自己的通道命令，可由CPU执行相应指令来启动通道，并在操作结束时向CPU发出中断信号。





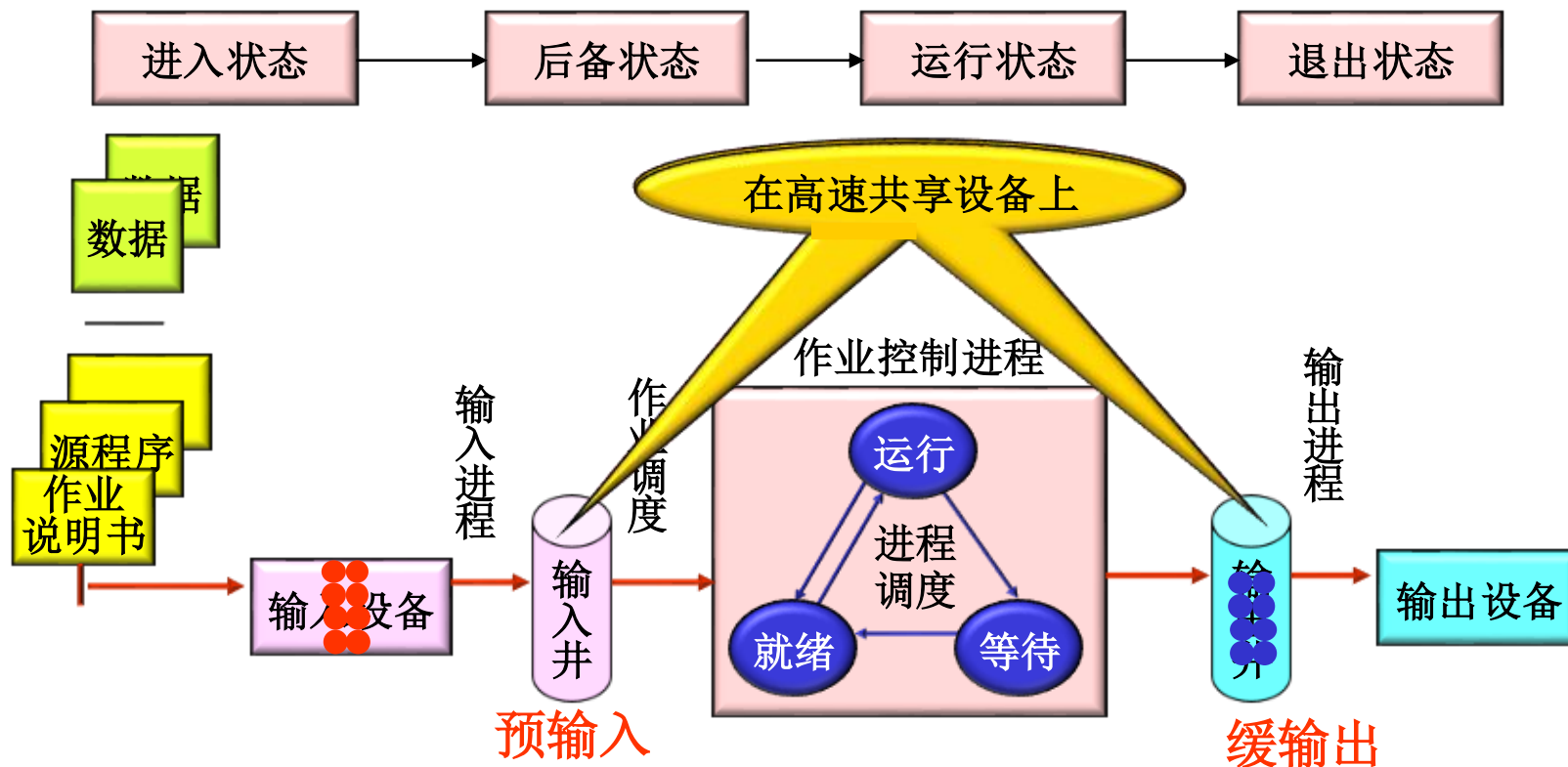
## 2.3 作业的建立



### 2.3.1 作业输入方式

#### (4) SPOOLING系统

##### ● SPOOLING系统作业和进程状态转换



● 使外设CPU直接控制下，与CPU并行工作（假脱机）





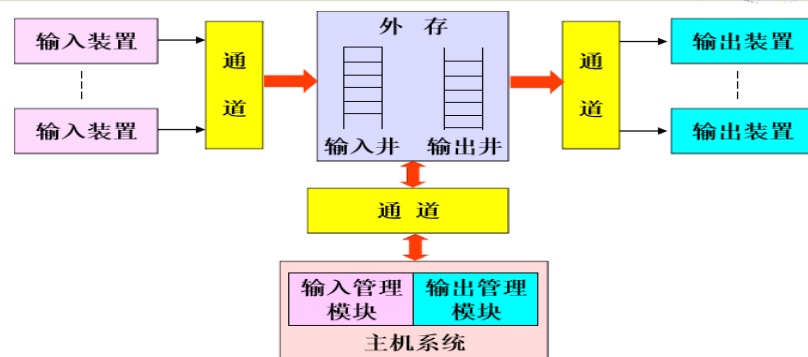
## 2.3 作业的建立



### 2.3.1 作业输入方式

#### (4) SPOOLING系统

##### ● SPOOLING系统工作原理



- ✓ 作业执行前，用慢速设备将作业预先输入到后援存储器（如磁盘、磁鼓，称为输入井）中，称为预输入
- ✓ 作业运行中，若需要使用数据时，从输入井中取出
- ✓ 作业执行中，不必直接启动外设输出数据，只需将这些数据写入输出井中
- ✓ 作业全部运行完毕，再启动外设输出全部数据和信息，称为缓输出

##### ● 实现了对作业输入、组织调度和输出的统一管理



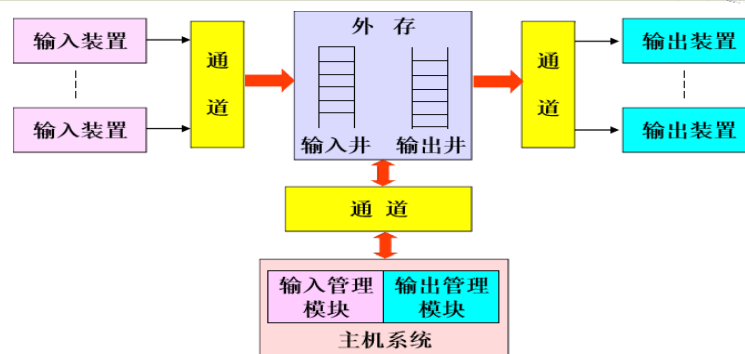
## 2.3 作业的建立



### 2.3.1 作业输入方式

#### (4) SPOOLING系统

● Spooling技术的实质是：



- ✓ 外围设备联机并行操作，是一种**速度匹配技术**，也是一种**虚拟设备技术**，
- ✓ 用一种高速**可共享的物理设备**模拟另一类**独享的物理设备**，使各作业在执行期间只使用虚拟的设备，而不直接使用独占的物理设备
- ✓ 可使**独占设备变成逻辑上可共享的设备**，使得设备的利用率和系统效率都能得到提高。



## 2.3 作业的建立



### 2.3.1 作业输入方式

#### (5) 网络输入方式

● **网络输入方式：**当用户需要在计算机网络中某一台主机上输入的信息传送到同一网络中的另一台主机上进行操作或执行时，即构成网络输入方式。

● **例如：**云计算中作业的提交





## 2.3 作业的建立



### 2.3.2 JCB的建立

- JCB是在作业建立时系统根据作业说明书建立的;
- 在运行过程中, JCB是系统对作业进行管理的信息
  - ✓ 作业名
  - ✓ 估计执行时间
  - ✓ 优先数 (用于调度)
  - ✓ 作业说明书文件名
  - ✓ 程序类型 (需调用的系统程序)
  - ✓ 资源要求 (静态, 或中间可以随作业步变化)
  - ✓ 作业状态 (提交、后备、执行、完成)



## 2.3 作业的建立



小结:

- 两阶段：作业输入、JCB创建
- 5种作业输入方式：
  - ✓ 联机
  - ✓ 脱机
  - ✓ 假脱机SPOOLing\*
  - ✓ 直接耦合
  - ✓ 网络方式



## 2.4 用户接口



- OS: 管理计算机资源, 目的在于 将计算机资源提供给用户使用, 用户通过**用户接口**使用计算机
- 用户接口是OS的五大功能之一, 为用户提供统一的接口是OS的目标之一。
- **用户接口的分类**
  - ① 程序级接口
    - API: 库函数-->系统调用
  - ② 命令级接口
    - 联机命令接口
    - 脱机命令接口
  - ③ 图形用户接口GUI



## 2.4 用户接口



### (1) 程序级接口

➤ 系统为用户在程序一级提供有关服务而设置，由一组系统调用或其封装函数组成

- ✓ 负责管理和控制运行的程序
- ✓ 在这些程序与系统控制的资源和提供的服务间实现交互作用

✓ 使用方式：

- ① 用汇编语言：在程序中直接用系统调用命令
- ② 用高级语言：可在编程时使用过程调用语句

以下代码段显示了系统调用sys\_write的使用

```
mov edx,4 ; message length
mov ecx,msg ; message to write
mov ebx,1 ; file descriptor (stdout)
mov eax,4 ; system call number (sys_write)
int 0x80 ; call kernel
```

```
#include<stdio.h>
#include<string.h>
#include<errno.h>
int main() {
 FILE* pf= fopen("test.txt", "w+");
 if (pf == NULL) {
 printf("%s\n", strerror(errno));
 return;
 }
 fputc('a', pf); //输入一个字符
 fclose(pf); //用完关闭文件
 pf = NULL;
 return 0;
}
```



## 2.4 用户接口



### (1) 程序级接口--系统调用分类

- **进程控制**: 创建和管理进程。例如`fork()`, `exec()`, `wait()`, `exit()`等。
- **文件管理**: 读写文件。例如`open()`、`read()`、`write()`、`close()`等函数。
- **设备管理**: 管理和控制设备。例如, `ioctl()`函数可以用于对设备进行各种控制。
- **信息维护**: 获取和设置系统数据。例如, `getpid()`可以获取进程的ID, `time()`可以获取当前的系统时间。
- **通信**: 处理进程间的通信。如, IPC机制(如信号、管道、消息队列、共享内存、信号量等)的`send()`、`receive()`等。
- **内存管理**: 管理内存资源。例如, `brk()`、`sbrk()`等函数可以用于改变进程的数据段大小(即, 分配或释放内存)。
- **网络管理**: 进行网络通信, 例如`socket()`、`bind()`、`connect()`、`listen()`、`accept()`等。





## 2.4 用户接口



### (2) 命令级接口——为用户提供各种命令

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows XP [版本 5.1.2600]
(C) 版权所有 1985-2001 Microsoft Corp.

C:\Documents and Settings\Administrator> dir

驱动器 c 中的卷是 系统盘
卷的序列号是 70EE-947D

C:\Documents and Settings\Administrator 的目录

2010-03-11 19:46 <DIR> .
2010-03-11 19:46 <DIR> ..
2009-10-22 12:08 <DIR> Favorites
2009-12-02 17:10 70 faxHeader.txt
2010-03-16 17:27 <DIR> My Documents
2009-05-21 16:07 <DIR> WINDOWS
2010-01-13 18:25 <DIR> 「开始」菜单
2010-03-18 08:12 <DIR> 桌面
 1 个文件 70 字节
 7 个目录 85,874,548,736 可用字节

C:\Documents and Settings\Administrator> g:

G:\>cd dm

G:\DM>
```



## 2.4 用户接口



### (2) 命令级接口——联机控制方式

- **环境设置**：改变终端用户所在位置、执行路径等；如cd、ls、pwd
- **执行权限管理**：控制用户访问系统和操作文件的权限；如chmod、chgrp、chattr
- **系统管理**：系统维护、开机关机、增加或减少终端用户、计时收费等；如shutdown、adduser、kill、su、passwd
- **文件管理**：管理和控制终端用户的文件；如find、grep
- **编辑、编译、链接装配和执行编辑命令**：gcc、ld、vi、Emacs
- **通信**：主机 $\longleftrightarrow$ 远程终端、主机 $\longleftrightarrow$ 主机；如telnet、talk、ping、ipconfig
- **资源要求**：用户向系统申请资源。

参考：B站“(北京交通大学)操作系统 [2-3-2]--ZGSOS[1-3-2]操作系统联机命令接口”



## 2.4 用户接口



### (2) 命令级接口

#### ● 输入输出重定向

- **定义：**输入输出重定向是**编程和命令行操作**中的一种功能，它允许将命令的输入和输出从默认的设备，通常是键盘和显示器，重定向到其他地方，如文件。
- **输入重定向：**指定一个文件或命令的输出作为新的输入设备。例如，  
`cat file.txt < /dev/null` 会将 `file.txt` 文件的内容导入到命令中，即使 `/dev/null` 通常被用作丢弃所有输入的设备。
- **输出重定向：**将命令的输出写入到指定的文件中。例如，  
`ls -l *.txt > output.txt` 会将 `ls -l *.txt` 的输出写入到 `output.txt` 文件中。
- **错误输出重定向：**它将命令的错误信息写入到指定的文件中。例如，  
`ls -l *.txt 2> error.txt` 会将 `ls -l *.txt` 的错误信息写入到 `error.txt` 文件中。

#### ● 输入输出重定向和管道

- 统计当前目录中有多少个文件：`ls -l | grep "^-" | wc -l`
- 统计当前目录中有多少个子目录：`ls -l | grep "^d" | wc -l`

#### ● 想想：将一个文件夹下的所有文件名放到文档中，怎么办？



## 2.4 用户接口



### (3) 图形用户接口 (GUI, Graphic User Interface)

- 在命令行方式下，用户与操作系统的交互要求用户**记忆命令格式**。
- 在图形用户接口方式下，用户可利用鼠标对屏幕上的**图标进行操作**，完成与操作系统的交互，从而减少记忆内容，方便用户使用。它的技术基础是**高分辨显示器和鼠标**。
- 70年代，美国施乐公司的研究人员开发出了第一个图形用户界面，实现了计算机使用从**字符界面向图形界面**的转变，开启了新的纪元。
  - 人机交互性、**美观性**、实用性、技术性

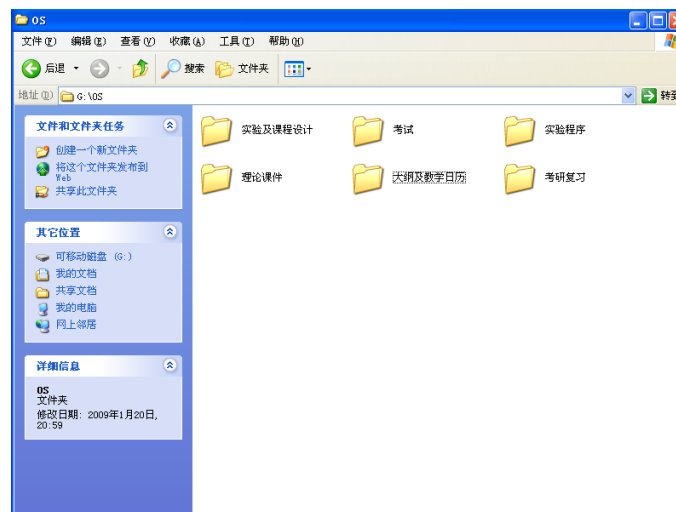
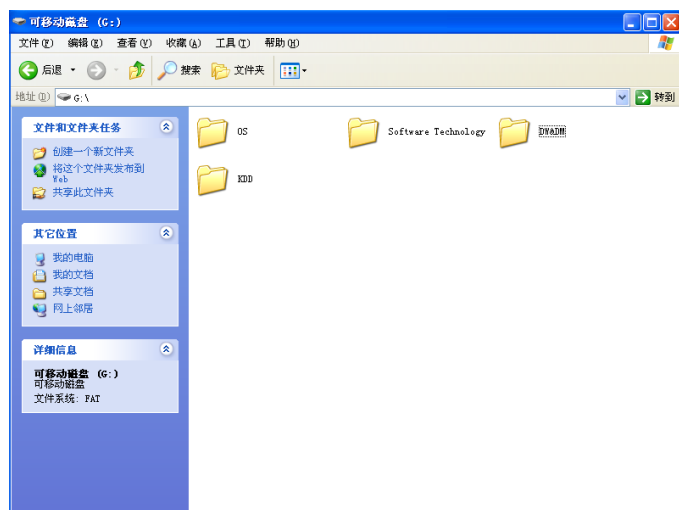


## 2.4 用户接口



### (3) 图形用户接口 (GUI, GRAPHIC USER INTERFACE)

- **窗口 (Window)** 是屏幕上的一块矩形区域，应用程序通过窗口向用户展示系统所提供的各种服务及其需要用户输入的信息。窗口界面上有标题条、控制**菜单框**、**菜单栏**、**滚动条**、**控制按钮**等；
- **图标 (icon)** 是代表一个应用程序的特殊的小位图，也是最小化的窗口，通过对图标的操作可以激活相应的程序或启动应用程序，包括：应用程序图标、组图标、应用程序项图标。
- **对话框 (Dialogue)**：类似窗口，但是大小不可调





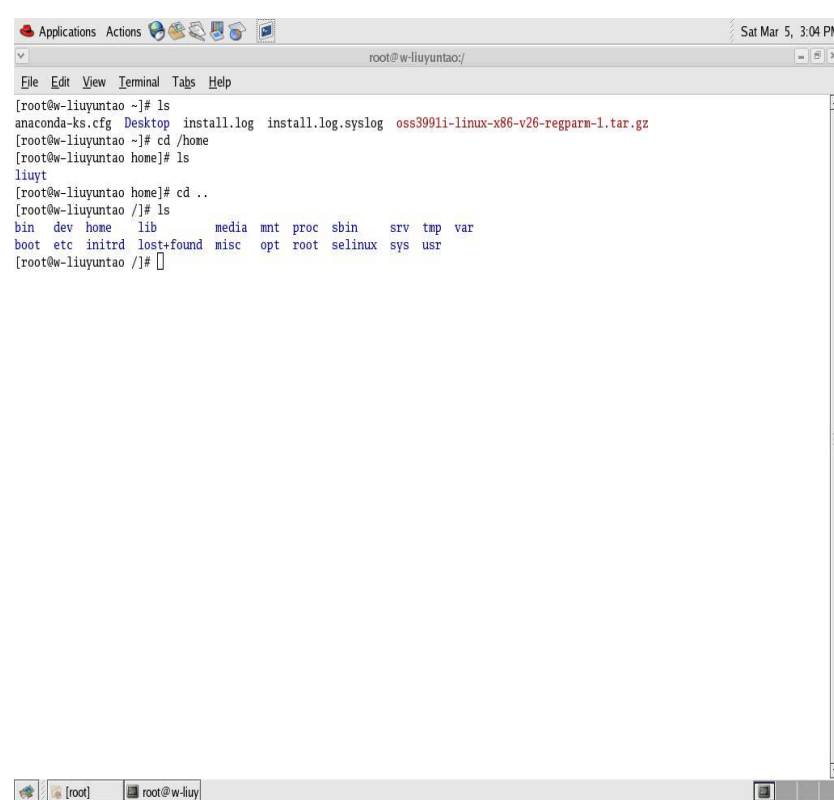
## 2.4 用户接口



### Linux的GUI窗口示例



### Linux命令行终端

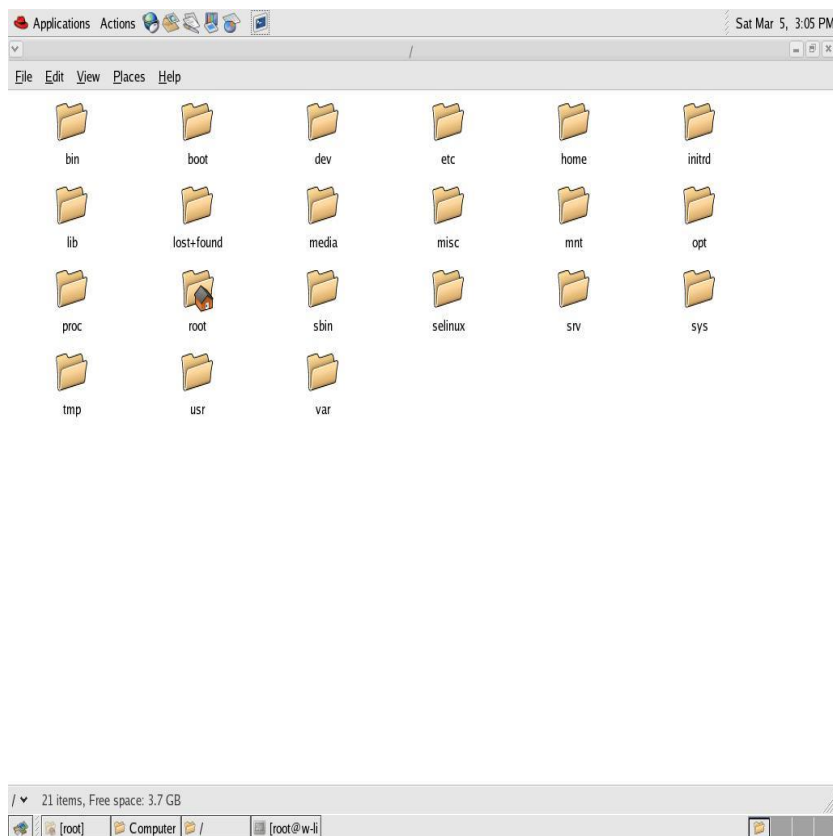




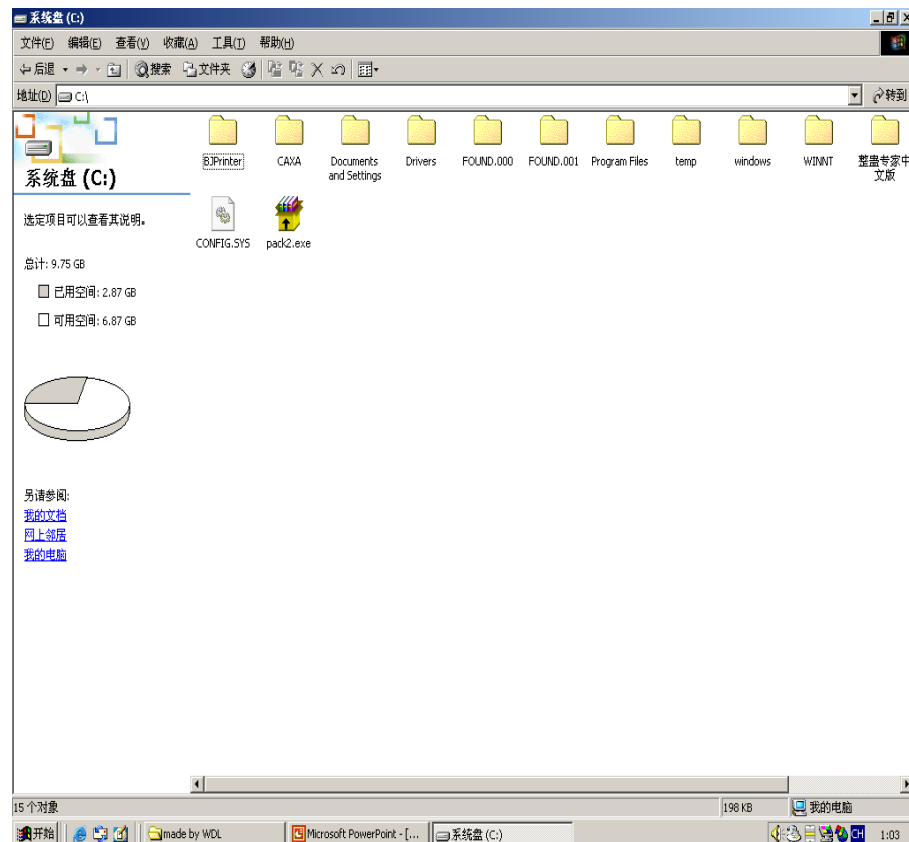
## 2.4 用户接口



### Linux图形用户接口



### Windows图形用户接口

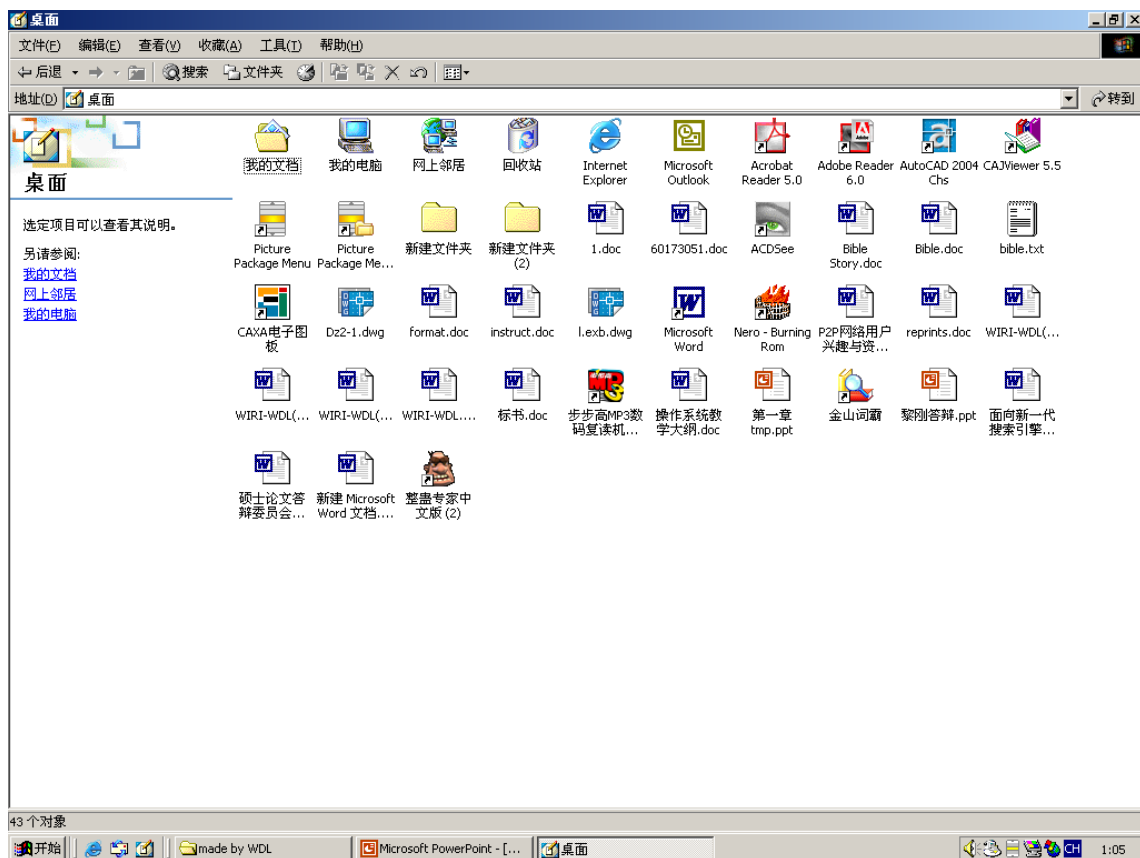




## 2.4 用户接口



### Windows图形用户接口--窗口



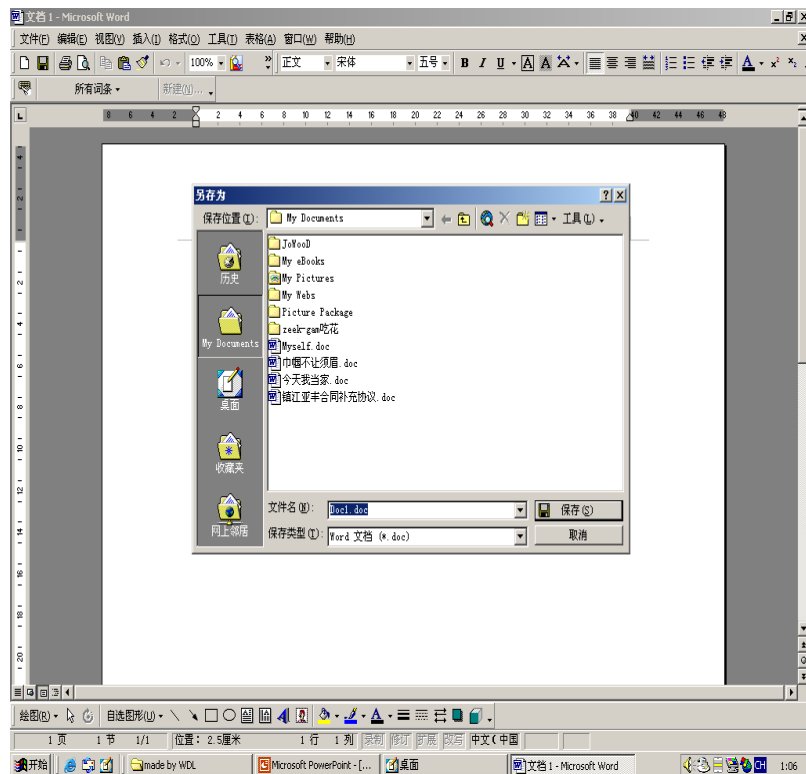




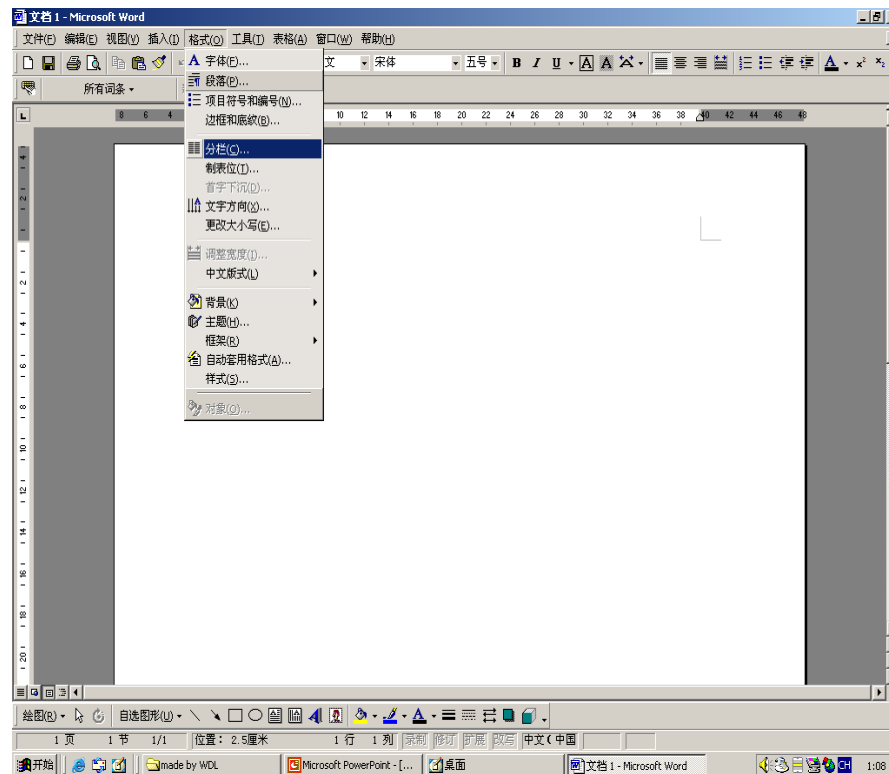
## 2.4 用户接口



### Windows GUI--对话框



### Windows GUI--菜单





## 2.4 用户接口



### 小结:

- 用户接口分为:
  - 程序级接口: 编写程序时使用
  - 命令接口: 需要记忆, 但快捷
  - GUI: 无需记忆, 需要鼠标支持, 显示设计美学
- Q&A



# Review4



1. 程序启动与结束：
  - 启动方式：命令行运行、批处理方式、exec调用、自启动运行
  - 结束：正常结束、非正常结束
2. 作业、作业步概念
3. **组织** = 程序 + 数据 + 作业说明书
4. 作业建立 = 作业输入 + JCB建立
5. 作业状态：输入（提交）、收容（后备）、运行、退出
6. 作业输入：联机、脱机、直接耦合、Spooling、网络方式
7. **Spooling\***：假脱机（外设同时联机操作）原理、用高速共享设备模拟独享设备的操作、实现独享设备的共享、实现预输入缓输出
8. 用户接口：命令级接口、GUI接口、程序级接口

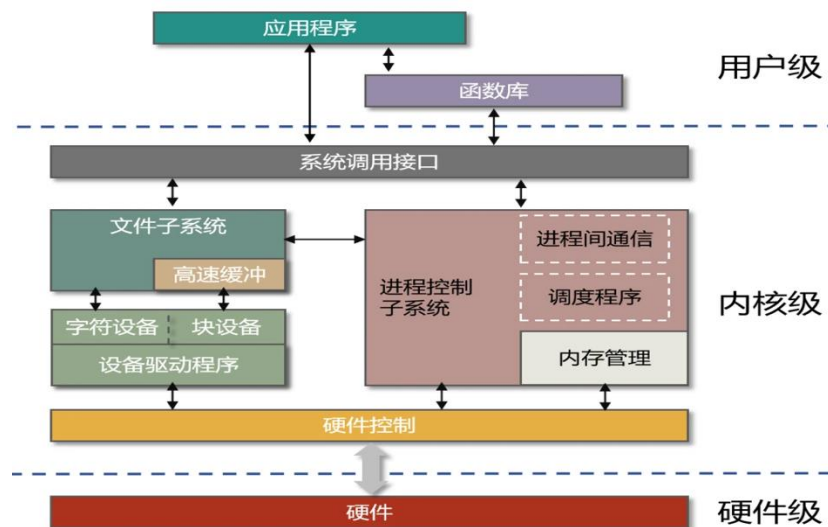


## 2.5 系统调用



### 2.5.1 系统调用概述

- 系统调用（System Call）—内核的出口
  - 系统调用：OS提供给用户程序调用的一组“特殊”接口。
  - 从逻辑上来说，系统调用可被看成是一个内核与用户程序交互的接口，把用户进程的请求传达给内核，待内核把请求处理完毕后再将处理结果送回给用户空间。
- 问题：
  - 用户如何使用系统调用？
  - 系统调用与API的关系？
  - OS如何提供系统调用？
  - 系统调用如何实现？
- 目标：SysCall原理、使用





## 2.5 系统调用



- 系统调用是OS提供给**软件开发人员的唯一接口**，开发人员可利用它使用系统功能。OS内核中有一组实现**系统调用功能**的过程。

- **程序级接口**

- 进程控制
- 进程通信
- 内存管理
- 文件管理
- 设备管理
- 信息维护
- 网络管理
- .....

```
void main() {
 int i;
 if (fork() == 0) {
 for (i = 1; i < 1000; i ++)
 printf("This is child process\n");
 }
 else {
 for (i = 1; i < 1000; i ++)
 printf("This is parent process\n");
 }
}
```

执行系统调用



## 2.5 系统调用



### 2.5.2 系统调用分类

#### ● 设备管理：设备的读写和控制

- ✓ ioctl 设备配置
- ✓ open 设备打开
- ✓ close 设备关闭
- ✓ read 读设备
- ✓ write 写设备



```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
int main()
{
 umask(0); //将umask置0
 //相当于C语言的fopen的"w", 只写、自动创建、自动覆盖
 int fd = open("log.txt",
 O_WRONLY | O_CREAT | O_TRUNC, 0666);
 if(fd < 0) return 1;

 char* msg = "hello Linux\n";
 //像fd描述的文件写入msg指向的字符串
 write(fd, msg, strlen(msg));
 close(fd);
 return 0;
}
```



## 2.5 系统调用



### 2.5.2 系统调用分类

#### ● 文件管理：文件读写和文件控制

- ✓ **open** 文件打开
- ✓ **close** 文件关闭
- ✓ **read** 读文件
- ✓ **write** 写文件
- ✓ **seek** 读写指针定位
- ✓ **creat** 文件创建
- ✓ **stat** 读文件状态
- ✓ **mount** 安装文件系统
- ✓ **chmod** 修改文件属性

#### ● 问题（Q&A）

- ✓ 怎么跟设备管理的SC重复？

从二进制文件读取结构体数据

```
#include <stdio.h>
#include <stdlib.h>
typedef struct {
 int id;
 char name[20];
} Student;
int main() {
 FILE *fp;
 Student s;
 // 以二进制读取模式打开文件
 fp = fopen("students.bin", "rb");
 if (fp != NULL) {
 while (fread(&s, sizeof(Student), 1, fp) == 1) {
 printf("ID: %d, Name: %s\n", s.id, s.name);
 }
 fclose(fp);
 } else {
 perror("打开文件时出错");
 return 1;
 }
 return 0;
}
```



## 2.5 系统调用



### 2.5.2 系统调用分类

- **进程控制**：创建、中止、暂停等控制
  - ✓ **fork** 创建进程
  - ✓ **exit** 进程自我终止
  - ✓ **wait** 阻塞当前进程
  - ✓ **sleep** 进程睡眠
  - ✓ **getpid** 获得进程标识



```
1 #include<stdio.h>
2 #include<unistd.h>
3 #include<sys/types.h>
4 #include<sys/wait.h>
5 #include<stdlib.h>
6
7 int main() {
8 pid_t id = fork();
9 if(id == 0){ //child
10 int count = 5;
11 while (count--){
12 printf("I am child...PID:%d, PPID:%d\n", getpid(), getppid());
13 sleep(1);
14 }
15 exit(10);
16 }
17 //father
18 int status = 0;
19 pid_t ret = waitpid(id, &status, 0);
20 if (ret >= 0){
21 //wait success
22 printf("wait success...\n");
23 if (WIFEXITED(status)){
24 //exit normal
25 printf("exit code:%d\n", WEXITSTATUS(status));
26 }
27 else{
28 //signal killed
29 printf("signal %d\n", status & 0x7F);
30 }
31 }
32 printf("father done ...\n");
33 sleep(3);
34 return 0;
35 }
```





## 2.5 系统调用



### 2.5.2 系统调用分类

#### ● 进程通信：进程之间传递消息或信号

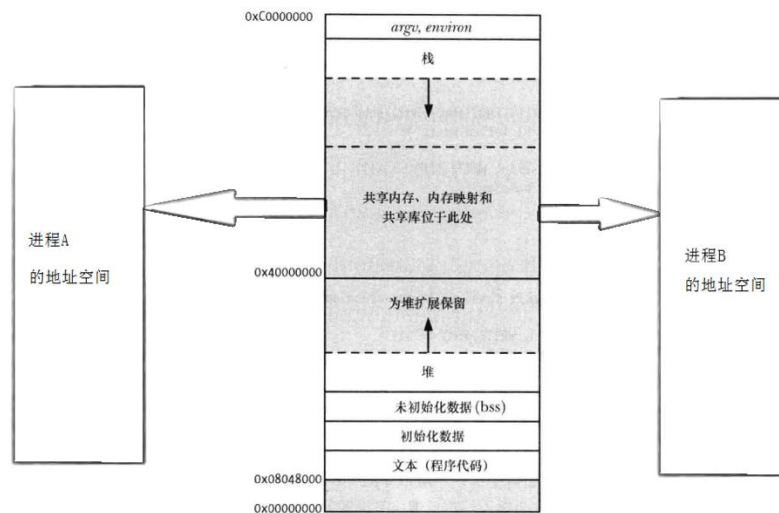
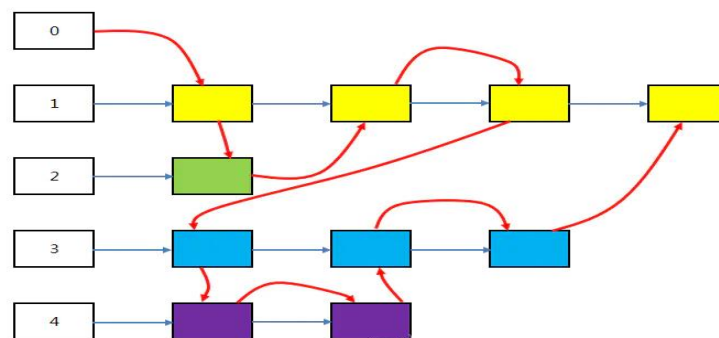
##### ✓ 消息队列：

- `msgsnd`：发送消息
- `msgrcv`：接收消息
- `msgget`：获取消息

##### ✓ 共享存储区：

- `shmget`：创建共享内存
- `shmat`：关联共享内存
- `shmdt`：解除共享内存
- `shmctl`：释放共享内存

##### ✓ `socket`等通信渠道的建立、使用和删除





## 2.5 系统调用



### 2.5.2 系统调用分类

#### ● 存储管理：内存的申请和释放

- **alloc**: 向**栈**申请内存
- **malloc**: 向**堆**申请内存;
- **free**: 释放内存
- ...

#### ● 系统管理：设置和读取时间、 读取用户和主机标识等

- **gtime** 读取时间
- **stime** 设置时间
- **getuid** 读取用户标识

```
#include<stdio.h>
#include"menu.h"
#include<stdlib.h>//memset
#include<string.h>//strcpy

int main()
{
 MENU_T *pmenu = NULL;
 //动态开空间
 pmenu = (MENU_T *)malloc(sizeof(MENU_T));
 printf("111\n");//111
 //开空间是否成功
 if(pmenu==NULL) {
 printf("open space fail\n");
 exit(0);
 }
 printf("open space success\n");
 memset(pmenu,'\0',sizeof(MENU_T));

 //操作创建的内存空间
 pmenu->menu_id = 1002;
 strcpy(pmenu->name,"黄焖鸡");
 pmenu->price=16.5;
 free(pmenu);
 printf("222\n");//222
 return 0;
}
```



## 2.5 系统调用



### 2.5.3 操作系统的POSIX 标准

- **问题（Q&A）：**
  - 应该提供哪些系统调用？参数如何设置？
  - 如何实现源代码级应用程序的**可移植性**？
- **POSIX 标准**
  - **POSIX**, Portable Operating System Interface for Computing Systems
  - 是由IEEE和ISO/IEC 开发的一簇标准。
  - 该标准是基于的**UNIX** 实践和经验，描述了OS的调用服务接口，用于保证编制的应用程序可以在**源代码**级上在多种OS上移植运行。
  - **Linux**在设计**Linux** 之初就意识到要遵循**POSIX**标准
- **遵循规范很重要，制定且执行自己的标准更重要！！**



## 2.5 系统调用



### 2.5.4 系统调用的实现过程

- 实际上系统调用语句本身是硬件提供的（机器指令），但其所调用的功能是OS提供的。每种机器的机器指令集中都有一条系统调用指令。

```
_start: ;User prompt
 mov eax, 4
 mov ebx, 1
 mov ecx, userMsg
 mov edx, lenUserMsg
 int 80h

 ;Read and store the user input
 mov eax, 3
 mov ebx, 2
 mov ecx, num
 mov edx, 5 ;5 bytes (num
 int 80h

 ;Output the message 'The entered
 mov eax, 4
 mov ebx, 1
 mov ecx, dispMsg
 mov edx, lenDispMsg
 int 80h
```

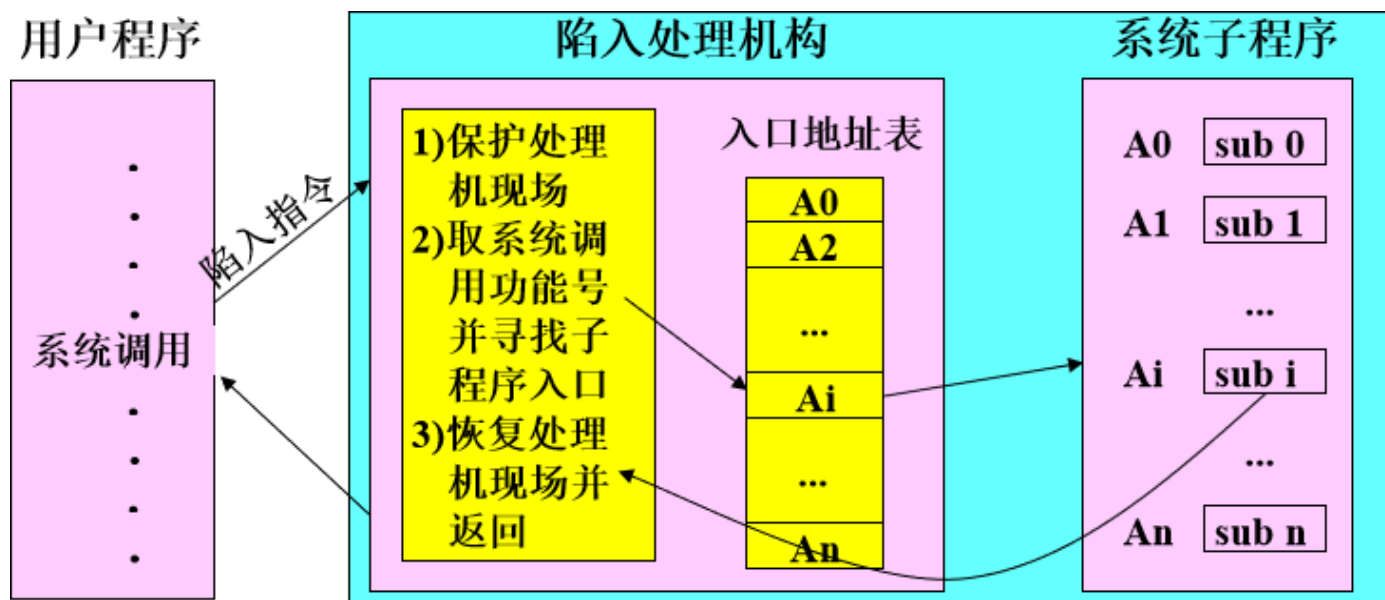


## 2.5 系统调用



### 2.5.4 系统调用的实现过程

#### ● 系统调用的处理流程

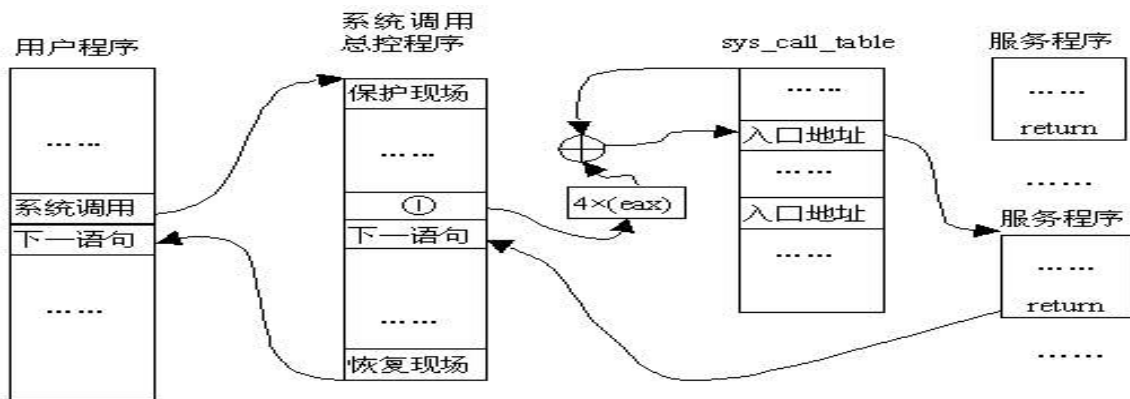
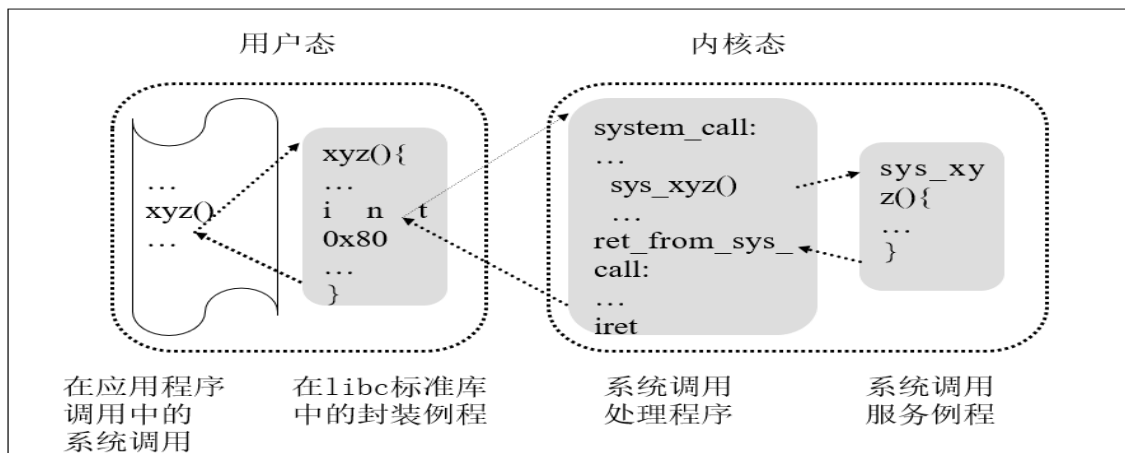




## 2.5 系统调用

### 2.5.4 系统调用的实现过程

#### 系统调用的处理流程



注①：此处语句为：`call *SYMBOL_NAME(sys_call_table)(,%eax,4);` eax 中为系统调用号

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
int main()
{
 umask(0); //将umask置0
 int fd = open("log.txt",
 O_WRONLY |
 O_CREAT |
 O_TRUNC,0666);

 if(fd < 0) return 1;
 char* msg = "hello Linux\n";

 write(fd,msg,strlen(msg));
 close(fd);
 return 0;
}
```



## 2.5 系统调用



### 2.5.5 系统调用与普通过程调用

#### ● 相同点

- ① 改变指令流程
- ② 重复执行和公用
- ③ 执行后需要返回调用处的下一条指令位置

#### ● 不同点

- ① 运行在不同的系统状态
  - 一般**过程调用**，其调用程序和被调用程序都运行在相同状态；
  - 而**系统调用**，调用程序在用户态，被调用程序运行在内核态；
- ② 通过软中断进入
  - 一般调用过程通过**过程调用语句**直接由调用过程转向被调用过程；
  - 而系统调用必须通过**系统调用指令**，由**软中断转向相应处理程序**，CPU由用户态转为核心态
- ③ 返回问题
  - 一般调用过程在被调用过程执行完毕后，**直接返回调用过程**；
  - 系统调用，在被调用过程执行完毕后，**做后处理**，判断是否做进程调度



## 2.5 系统调用



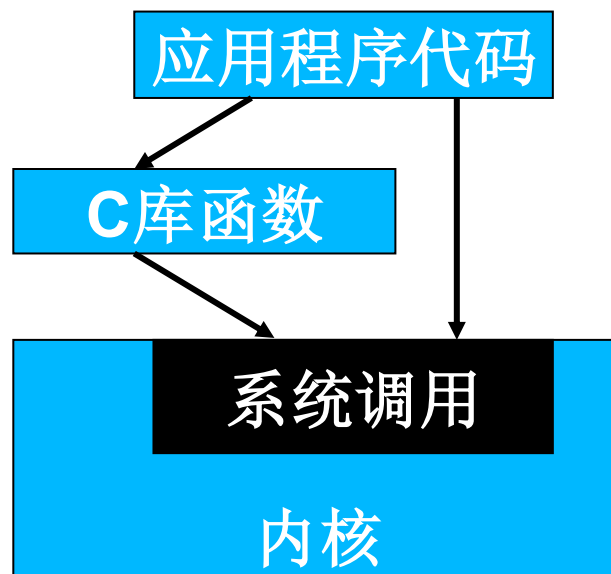
### 2.5.5 系统调用和API

#### ❖ 系统调用和应用编程接口(API)是不同的

- **APIs**是一组库函数定义, 说明如何获得一个给定的服务
- **系统调用**通过**软中断**向内核发出一个明确的请求, 例如windows是 INT 21H, Linux是INT 80H
- API有可能和系统调用的调用形式一致

#### ❖ Libc库定义的一些API, 引用了封装例程(wrapper routine, 目的就是发布系统调用)

- 一般每个系统调用对应一个封装例程
- Libc库再用这些封装例程定义出给用户的API







# 应用程序、函数库系统调用间的关系

```
#include <unistd.h>
#include <stdio.h>
int main(int argc, void ** argv) {
 int pid = fork();
 if(pid == -1) {
 printf("error!");
 } else if(pid == 0) {
 printf("This is the child process!");
 } else {
 printf("This is the parent process! child process id
 = %d", pid); }
 return 0;
}
```

问题：这里的**fork()**是什么？  
系统调用，or API（或者是  
**libc**库函数）



# 利用系统调用复制文件



```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <fcntl.h>
#include <unistd.h>
#define BUF_SIZE 4096
#define OUTPUT_MODE 0700
int main(int argc, char *argv[]) {
 int in_fd, out_fd, rd_count, wt_count;
 char buffer[BUF_SIZE];
 if(argc != 3) exit(1);
 in_fd = open(argv[1], O_RDONLY);
 if(in_fd < 0) exit(2);
 out_fd = creat(argv[2], OUTPUT_MODE);
 if(out_fd < 0) exit(3);
```

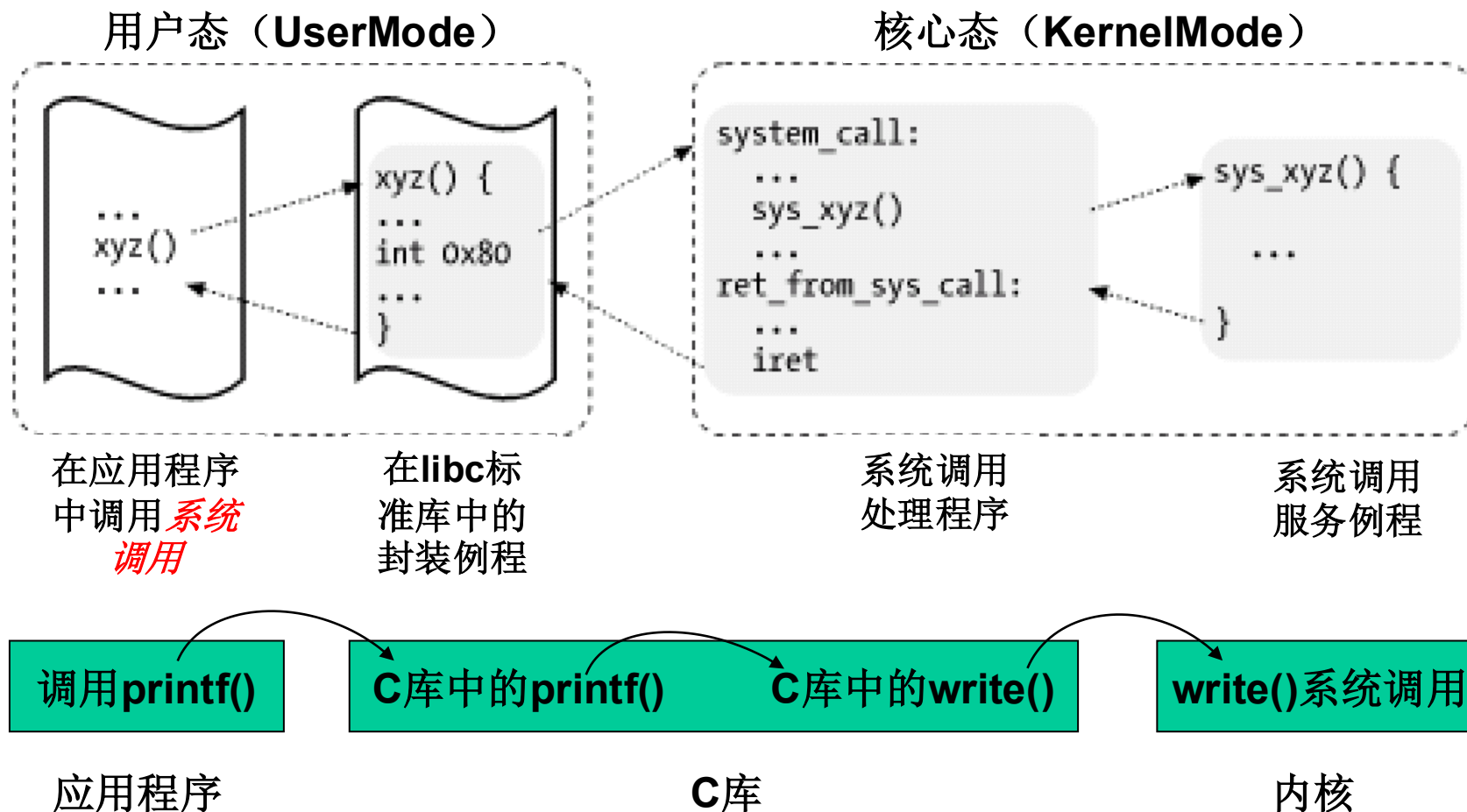
(续)

```
while(1)
{
 rd_count = read(in_fd, buffer,
 BUF_SIZE);
 if(rd_count <= 0) break;
 wt_count = write(out_fd, buffer,
 rd_count);
 if(wt_count <= 0) exit(4);
}
close(in_fd);
close(out_fd);
if(rd_count == 0) exit(0);
else exit(5);
return 0; }
```



## 2.5 系统调用

### 应用程序、封装例程、系统调用处理程序及系统调用服务例程间的关系





## 2.5 系统调用



### 2.5.6 Linux(openEuler)系统调用简介

- Linux系统有几百个系统调用（L2.6.26有326个, L3.0 347个）
- 如何标识一个系统调用？
  - 系统调用名？
  - 系统调用号？
  - .....？



## 2.5 系统调用-Linux(openEuler)SC简介



### (1) 系统调用的数据结构--系统调用号

- **服务例程的命名**: Linux中系统调用的服务例程是以“**sys\_**”为前缀, 例如“**sys\_exit**”是进程退出的服务例程。
- **系统调用号**: 为了唯一标识每一个系统调用, Linux为**每个系统调用**分配一个**唯一的编号**, 称为**系统调用号**
- **系统调用号的宏定义**: 是将系统调用函数中的“**sys\_**”前缀换成“**\_\_NR\_**”前缀构成, 方便用户编程使用。
- **系统调用号很关键**: 一旦分配好不能再改变, 否则编译好的应用程序就会因调用到错误的系统调用而导致程序崩溃
- 在文件linux/arch/x86/include/asm/unistd\_32.h中
- **问题**: **系统调用号**与其**宏定义**的关系是什么?

```
/* This file contains the sys. call numbers*/
#define __NR_restart_syscall 0
#define __NR_exit 1
#define __NR_fork 2
#define __NR_read 3
#define __NR_write 4
#define __NR_open 5
.....
#define __NR_timerfd_gettime 326
```



## 2.5 系统调用-Linux(openEuler)系统调用简介

### (2) 系统调用的数据结构-系统调用表

❖ **问题：**如何找到每个系统调用号对应的服务例程？

❖ **方案：**

- 设置**系统调用分派表(dispatch table)**，把系统调用号与相应的服务例程关联起来，存放在**sys\_call\_table**数组中，有若干个表项(2.6.26中是326个)
- 第n个表项对应了n号系统调用的服务例程入口地址的指针，此时，**系统调用号**是系统调用服务例程在**系统调用表**中的下标

❖ **观察sys\_call\_table**

(syscall\_table\_32.S以及entry\_32.S最后)

#### ENTRY(sys\_call\_table)

```
.long sys_restart_syscall /* 0 - old
"setup()" sys. call, used for restarting */
.long sys_exit
.long sys_fork
.long sys_read
.long sys_write
.long sys_open /* 5 */
.long sys_close
.....
```

linux-2.6.26.arch.x86.kernel.syscall\_table\_32.S



## 2.5 系统调用-Linux(openEuler)系统调用

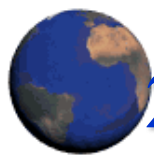


### 系统调用号、宏定义名和服务例程对应表

| 系统调用号 | 宏定义名                 | 系统调用服务例程            | 功能      |
|-------|----------------------|---------------------|---------|
| 0     | __NR_restart_syscall | sys_restart_syscall | 重启      |
| 1     | __NR_exit            | sys_exit            | 终止进程    |
| 2     | __NR_fork            | sys_fork            | 创建进程    |
| 3     | __NR_read            | sys_read            | 读文件     |
| 4     | __NR_write           | sys_write           | 写文件     |
| 5     | __NR_open            | sys_open            | 打开文件    |
| 6     | __NR_close           | sys_close           | 关闭文件    |
| 7     | __NR_waitpid         | sys_waitpid         | 等待子进程结束 |
| ...   | .....                | .....               |         |

涉及了两张表:

- 系统调用宏定义表 `unistd.h`
- 系统调用服务例程入口地址表 `sys_call_table` ( `syscall_table_32.S` )

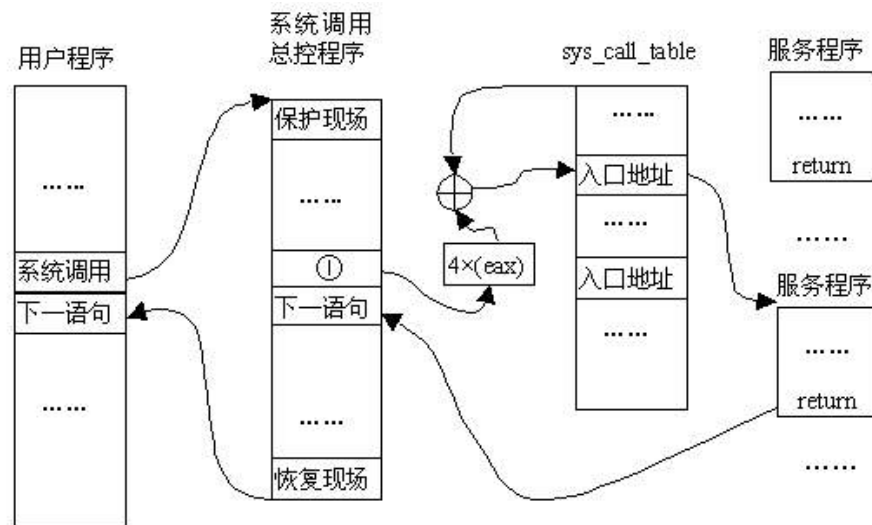


## 2.5 系统调用-Linux(openEuler)系统调用简介

### (3) 系统调用处理程序-- system\_call()函数

#### ➤ system\_call()函数：系统调用处理程序

1. **SAVE\_ALL**: 在进程的内核态堆栈中保存大多数寄存器的内容(即保存恢复进程到用户态执行所需要的上下文)
2. 对用户态进程传递来的系统调用号进行有效性检查，调用号不能超过nr\_syscalls
3. 根据EAX中的系统调用号调用对应的系统调用服务例程，  
`call *sys_call_table(,%eax,4)`，  
即sys\_xxx
4. 系统调用返回处理：
  - EAX保存返回值，
  - 跳转到syscall\_exit()



注①：此处语句为：`call *SYMBOL_NAME(sys_call_table)(,%eax,4)`；eax 中为系统调用号







### (3) 系统调用处理程序--`system_call()`函数

## restore\_all:

```
movl PT_EFLAGS(%esp), %eax # mix EFLAGS, SS and CS
```

**# Warning: PT\_OLDSS(%esp) contains the wrong/random values if we are returning to the kernel.**

**# See comments in process.c:copy\_thread() for details.**

**movb PT\_OLDSS(%esp), %ah****movb PT CS(%esp), %al**

**andl \$(X86\_EFLAGS\_VM | (SEGMENT TI MASK << 8) | SEGMENT RPL MASK), %eax**

```
cmpl $((SEGMENT LDT << 8) | USER RPL), %eax
```

## CFI REMEMBER STATE

```
je ldt ss # returning to user-space with LDT SS
```

## restore nocheck:

**TRACE IRQS IRET**

**restore nocheck notrace:**

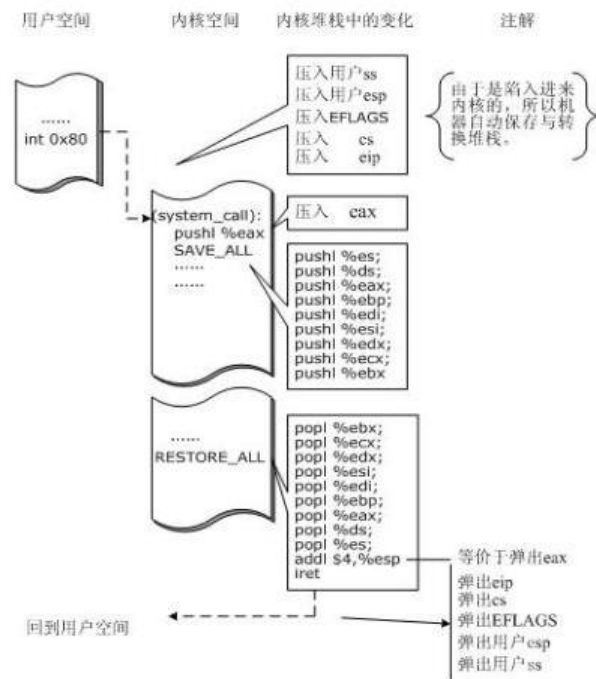
## RESTORE REGS

```
addl $4, %esp # skip orig_eax/error_code
```

CFI ADJUST CFA OFFSET -4

**irq return:**

## INTERRUPT RETURN





## 2.5 系统调用-Linux系统调用简介

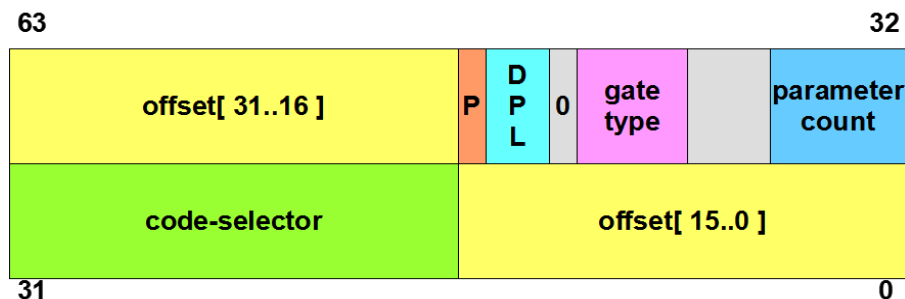
### (4) 系统调用初始化

- ❖ 内核初始化期间start\_kernel函数中调用trap\_init()函数建立IDT表中向量128对应的表项，语句如下：

```
set_system_gate(SYSCALL_VECTOR, &system_call);
```

```
#define SYSCALL_VECTOR 0x80
```

- ❖ 该调用把下列值存入这个系统门描述符的相应字段：
  - segment selector: 内核代码段\_\_KERNEL\_CS的段选择符
  - Offset: 指向哪里? 指向system\_call()异常处理程序的入口地址
  - Type: 置为15。表示这个异常是一个陷阱，相应的处理程序不禁止可屏蔽中断
  - DPL(描述符特权级)应该设置为多少?
    - DPL = 3





## 2.5 系统调用



### 小结

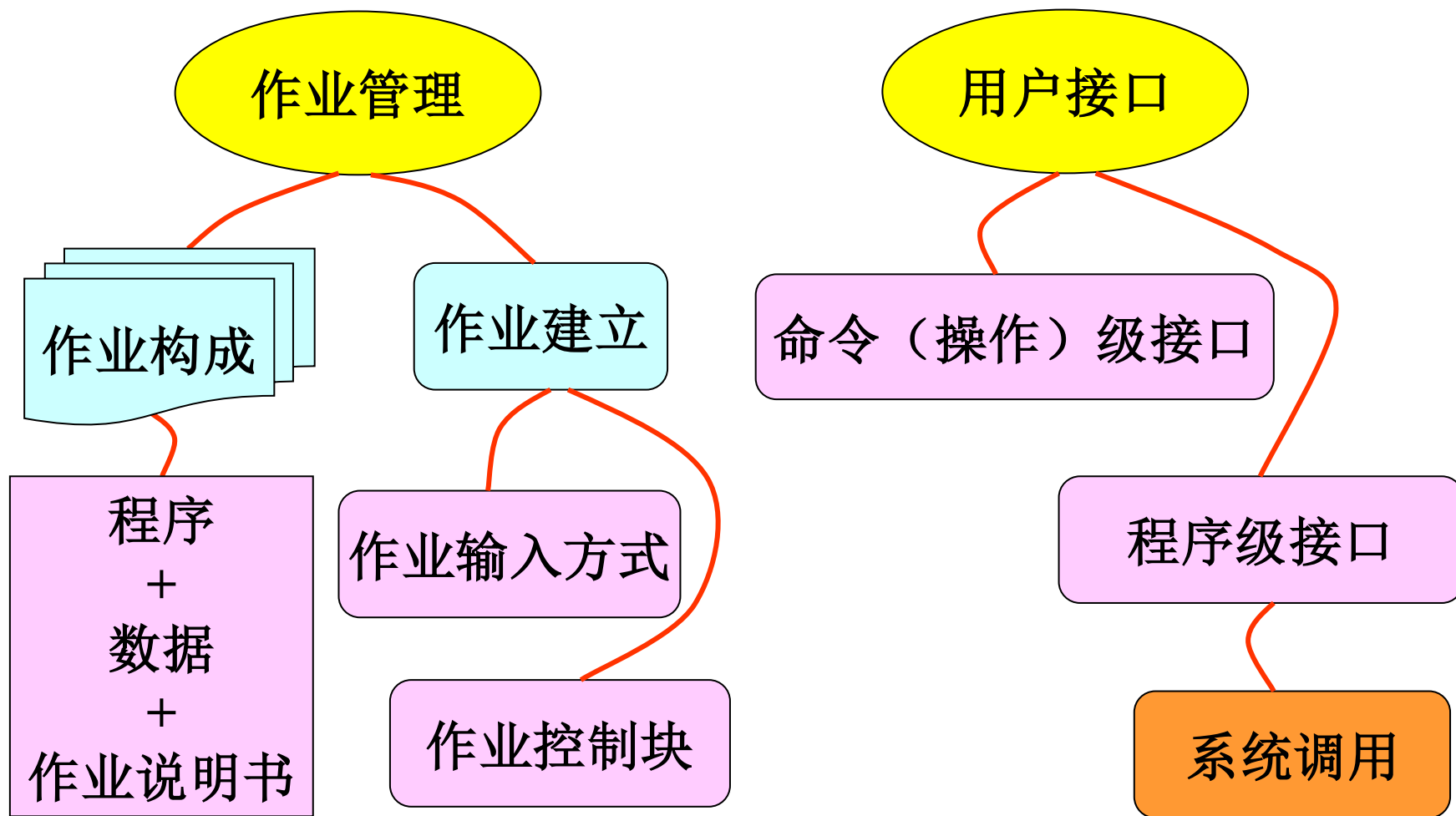
- 系统调用分类
- 系统调用处理流程\*
- 系统调用与API、函数调用的区别
- 接口标准: **POSIX**
- **Linux(openEuler)**的系统调用实现

### • 参考:

- 参考: B站“(北京交通大学)操作系统 [2-3-2]--ZGSOS[1-3-2]操作系统联机命令接口”



## 第二章小结





# 作业2



## 思考与实践：

1. 熟悉openEuler (Linux) 下的基本的文件操作系统、系统管理命令；
2. 学习openEuler (Linux) 下的c编程，主要是gcc、gdb工具的使用。
3. 为后续在openEuler (Linux) 下的编程实验做准备
4. 具体内容见“实验（实践）2-熟悉openEuler(Linux)系统命令和编程环境.docx”

详见作业2文档。



# 继续学习第3章： 进程管理

